

1. Personal Portfolio Website using HTML and CSS

Program Coding:

Index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Student Portfolio - Home</title>
  <style>
    body {
      margin: 0;
      font-family: 'Segoe UI', sans-serif;
      background-color: #f1f5f9;
    }
    /* Navigation Bar */
    nav {
      background-color: #0f172a;
      color: white;
      display: flex;
      justify-content: space-between;
      align-items: center;
      padding: 15px 60px;
    }
    nav h2 {
      margin: 0;
      font-size: 22px;
      letter-spacing: 1px;
    }
    nav ul {
      list-style: none;
      display: flex;
      gap: 25px;
      margin: 0;
      padding: 0;
    }
    nav ul li a {
      text-decoration: none;
      color: #cbd5e1;
      font-weight: 500;
      transition: 0.3s;
```

```

    }
    nav ul li a:hover {
      color: #2563eb;
    }
    /* Hero Section */
    .home {
      height: 90vh;
      display: flex;
      flex-direction: column;
      justify-content: center;
      align-items: center;
      text-align: center;
      background: linear-gradient(to right,
#1e293b, #0f172a);
      color: white;
    }
    .home h1 {
      font-size: 48px;
      margin-bottom: 10px;
    }
    .home p {
      font-size: 20px;
      color: #cbd5e1;
      margin-bottom: 30px;
    }
    .home button {
      padding: 12px 30px;
      font-size: 16px;
      background-color: #2563eb;
      color: white;
      border: none;
      border-radius: 6px;
      cursor: pointer;
      transition: 0.3s;
    }
    .home button:hover {
      background-color: #1d4ed8;
      transform: scale(1.05);
    }
  }
</style>
</head>
<body>
```

```

    }
  </style>
</head>
<body>
<nav>
  <h2>My Portfolio</h2>
  <ul>
    <li><a
href="index.html">Home</a></li>
    <li><a
href="professional.html">Professional
Info</a></li>
    <li><a
href="achievements.html">Achievements</
a></li>
    <li><a
href="experience.html">Experience</a></li>
  </ul>
</nav>
<section class="home">
  <h1>Hello, I'm Varshini selvakumar</h1>
  <p>Computer Science Student |
Front-End Developer</p>
  <button>Contact Me</button>
</section>
</body>
</html>

```

Professional.html

```

<!DOCTYPE html>
<html>
<head>
  <title>Professional Information</title>
  <style>
    body {
      margin: 0;
      font-family: 'Segoe UI', sans-serif;
      background-color: #f1f5f9;
    }
  </style>
  <nav>

```

```

  nav {
    background-color: #0f172a;
    color: white;
    display: flex;
    justify-content: space-between;
    align-items: center;
    padding: 15px 60px;
  }
  nav h2 {
    margin: 0;
    font-size: 22px;
    letter-spacing: 1px;
  }
  nav ul {
    list-style: none;
    display: flex;
    gap: 25px;
    margin: 0;
    padding: 0;
  }
  nav ul li a {
    text-decoration: none;
    color: #cbd5e1;
    font-weight: 500;
    transition: 0.3s;
  }
  nav ul li a:hover {
    color: #2563eb;
  }
  /* Page Title */
  .page-title {
    text-align: center;
    padding: 40px 20px 20px 20px;
    color: #0f172a;
  }
  .page-title h1 {
    font-size: 36px;
    margin-bottom: 10px;
  }
  /* Content Section */
  .container {

```

```

width: 80%;
margin: auto;
padding: 20px 0 60px 0;
display: grid;
grid-template-columns:
repeat(auto-fit, minmax(300px, 1fr));
gap: 25px;
}
.card {
background-color: white;
padding: 25px;
border-radius: 10px;
box-shadow: 0 5px 15px
rgba(0,0,0,0.1);
transition: 0.3s;
}
.card:hover {
transform: translateY(-5px);
}
.card h3 {
margin-top: 0;
color: #2563eb;
}
.card p, .card li {
color: #334155;
line-height: 1.6;
}
ul {
padding-left: 20px;
}
</style>
</head>
<body>
<nav>
<h2>My Portfolio</h2>
<ul>
<li><a
href="index.html">Home</a></li>
<li><a
href="professional.html">Professional
Info</a></li>

```

```

<li><a
href="achievements.html">Achievements</a></li>
<li><a
href="experience.html">Experience</a></li>
</ul>
</nav>
<div class="page-title">
<h1>Professional Information</h1>
</div>
<div class="container">
<div class="card">
<h3><img alt="graduation cap icon" data-bbox="610 348 635 363"/> Education</h3>
<p><strong>Bachelor in Computer
Science</strong></p>
<p>PSGR Krishnammal college for
women</p>
<p>2023 - 2026</p>
</div>
<div class="card">
<h3><img alt="laptop icon" data-bbox="610 508 635 523"/> Technical Skills</h3>
<ul>
<li>HTML & CSS</li>
<li>JavaScript</li>
<li>Python</li>
<li>C++</li>
</ul>
</div>
<div class="card">
<h3><img alt="target icon" data-bbox="610 688 635 703"/> Career Objective</h3>
<p>
Motivated Computer Science student
seeking opportunities
to apply technical skills in web
development and contribute
to innovative projects.
</p>
</div>
</div> </html>

```

Achievements.html:

```
<!DOCTYPE html>
<html>
<head>
  <title>Achievements</title>
  <style>
    body {
      margin: 0;
      font-family: 'Segoe UI', sans-serif;
      background-color: #f1f5f9;
    }
    /* Navigation Bar */
    nav {
      background-color: #0f172a;
      color: white;
      display: flex;
      justify-content: space-between;
      align-items: center;
      padding: 15px 60px;
    }
    nav h2 {
      margin: 0;
      font-size: 22px;
      letter-spacing: 1px;
    }
    nav ul {
      list-style: none;
      display: flex;
      gap: 25px;
      margin: 0;
      padding: 0;
    }
    nav ul li a {
      text-decoration: none;
      color: #cbd5e1;
      font-weight: 500;
      transition: 0.3s;
    }
    nav ul li a:hover {
      color: #2563eb;
    }
  </style>
</head>
<body>
```

```
    /* Page Title */
    .page-title {
      text-align: center;
      padding: 40px 20px 20px 20px;
      color: #0f172a;
    }
    .page-title h1 {
      font-size: 36px;
      margin-bottom: 10px;
    }
    /* Achievement Cards */
    .container {
      width: 80%;
      margin: auto;
      padding: 20px 0 60px 0;
      display: grid;
      grid-template-columns:
        repeat(auto-fit, minmax(300px, 1fr));
      gap: 25px;
    }
    .card {
      background-color: white;
      padding: 25px;
      border-radius: 10px;
      box-shadow: 0 5px 15px
        rgba(0,0,0,0.1);
      transition: 0.3s;
    }
    .card:hover {
      transform: translateY(-5px);
    }
    .card h3 {
      margin-top: 0;
      color: #2563eb;
    }
    .card p {
      color: #334155;
      line-height: 1.6;
    }
    .highlight {
      font-weight: bold;
    }
  </body>
</html>
```

```

        color: #0f172a;
    }
</style>
</head>
<body>
<nav>
    <h2>My Portfolio</h2>
    <ul>
        <li><a
href="index.html">Home</a></li>
        <li><a
href="professional.html">Professional
Info</a></li>
        <li><a
href="achievements.html">Achievements</
a></li>
        <li><a
href="experience.html">Experience</a></li>
    >
    </ul>
</nav>
<div class="page-title">
    <h1>My Achievements</h1>
</div>
<div class="container">
    <div class="card">
        <h3>🏆 Coding Competition
Winner</h3>
        <p>
            Secured <span class="highlight">1st
Prize</span> in Inter-College Coding
Competition 2024.
        </p>
    </div>
    <div class="card">
        <h3>📄 Web Development
Certification</h3>
        <p>
            Successfully completed a certified
course in Full Stack Web Development.
        </p>
    </div>

```

```

</div>
<div class="card">
    <h3>🎯 Academic Excellence</h3>
    <p>
        Achieved <span
class="highlight">95% marks</span> in
Final Year Project.
    </p>
</div>
<div class="card">
    <h3>🚀 Hackathon Participant</h3>
    <p>
        Participated in National Level
Hackathon and developed an innovative web
solution.
    </p>
</div>
</div>
</body>
</html>

```

Experience.html:

```

<!DOCTYPE html>
<html>
<head>
    <title>Experience</title>
    <style>
        body {
            margin: 0;
            font-family: 'Segoe UI', sans-serif;
            background-color: #f1f5f9;
        }
        /* Navigation Bar */
        nav {
            background-color: #0f172a;
            color: white;
            display: flex;
            justify-content: space-between;
            align-items: center;
            padding: 15px 60px;
        }
    </style>

```

```

nav h2 {
  margin: 0;
  font-size: 22px;
  letter-spacing: 1px;
}
nav ul {
  list-style: none;
  display: flex;
  gap: 25px;
  margin: 0;
  padding: 0;
}
nav ul li a {
  text-decoration: none;
  color: #cbd5e1;
  font-weight: 500;
  transition: 0.3s;
}
/*nav ul li a:hover {
  color: #2563eb;
}*/
/* Page Title */
.page-title {
  text-align: center;
  padding: 40px 20px 20px 20px;
  color: #0f172a;
}
.page-title h1 {
  font-size: 36px;
  margin-bottom: 10px;
}
/* Timeline Section */
.timeline {
  width: 80%;
  margin: 40px auto 80px auto;
  position: relative;
}
.timeline::after {
  content: "";
  position: absolute;
  width: 4px;

```

```

/*background-color: #0a0a0a;*/
top: 0;
bottom: 0;
left: 50%;
margin-left: -2px;
}
.container {
  padding: 10px 20px;
  position: relative;
  background-color: inherit;
  width: 50%;
}
.container.left {
  left: 0;
}
.container.right {
  left: 50%;
}
.content {
  padding: 10px;
  background-color: white;
  border-radius: 4px;
  box-shadow: 0 5px 15px
  rgba(0,0,0,0.1);
  transition: 0.3s;
}
.content:hover {
  transform: translateY(-5px);
}
.content h3 {
  margin-top: 0;
  color: #2563eb;
}
.content p {
  color: #334155;
  line-height: 1.6;
}
/* Responsive */
@media screen and (max-width:
768px) {
  .timeline::after {

```

```

        left: 31px;
    }
    .container {
        width: 75%;
        padding-left: 50px;
        padding-right: 10px;
    }
    .container.right {
        left: 0%;
    }
}
</style>
</head>
<body>
<nav>
    <h2>My Portfolio</h2>
    <ul>
        <li><a
href="index.html">Home</a></li>
        <li><a
href="professional.html">Professional
Info</a></li>
        <li><a
href="achievements.html">Achievements</
a></li>
        <li><a
href="experience.html">Experience</a></li>
    </ul>
</nav>
<div class="page-title">
    <h1>My Experience</h1>
</div>
<div class="timeline">

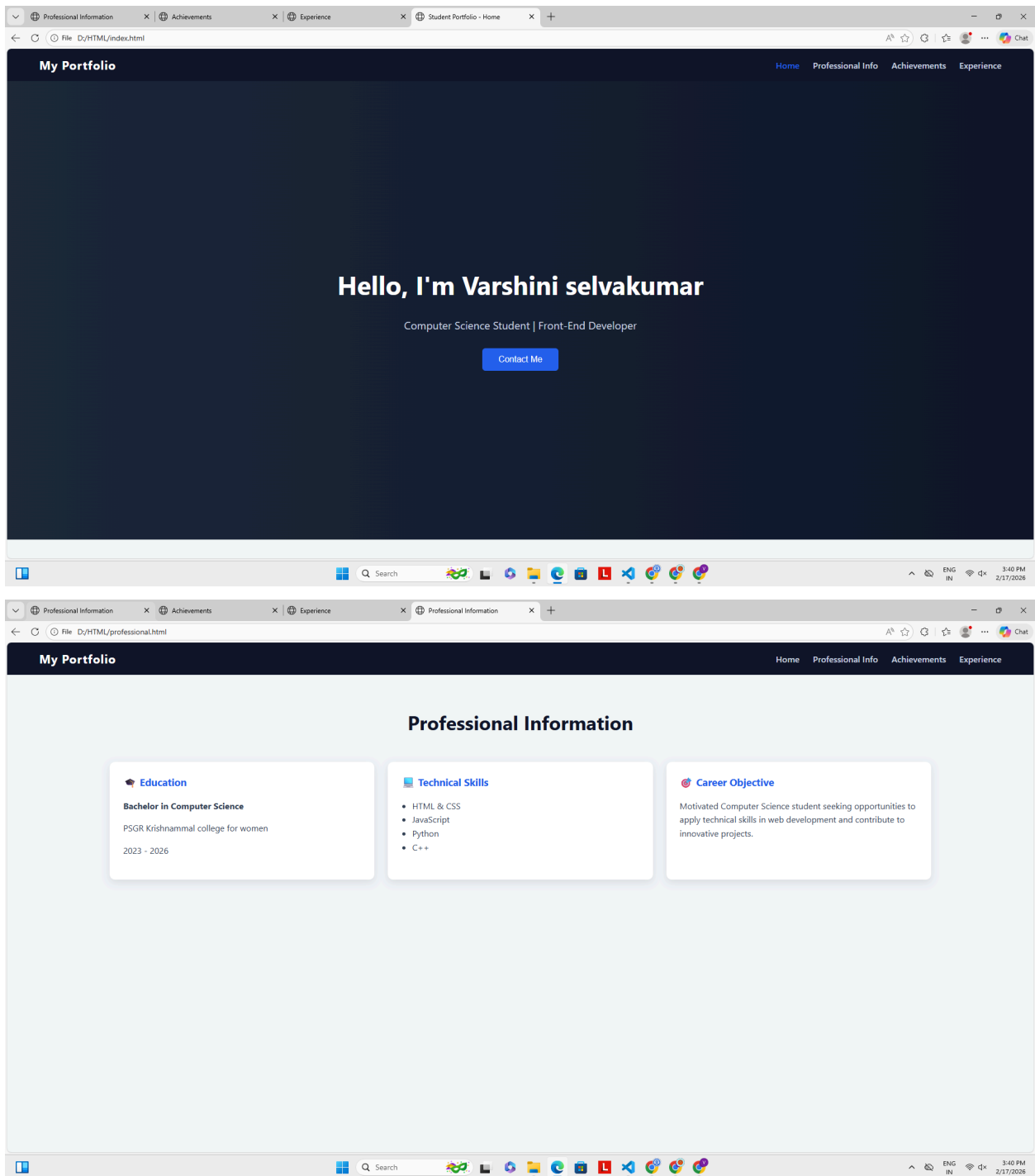
```

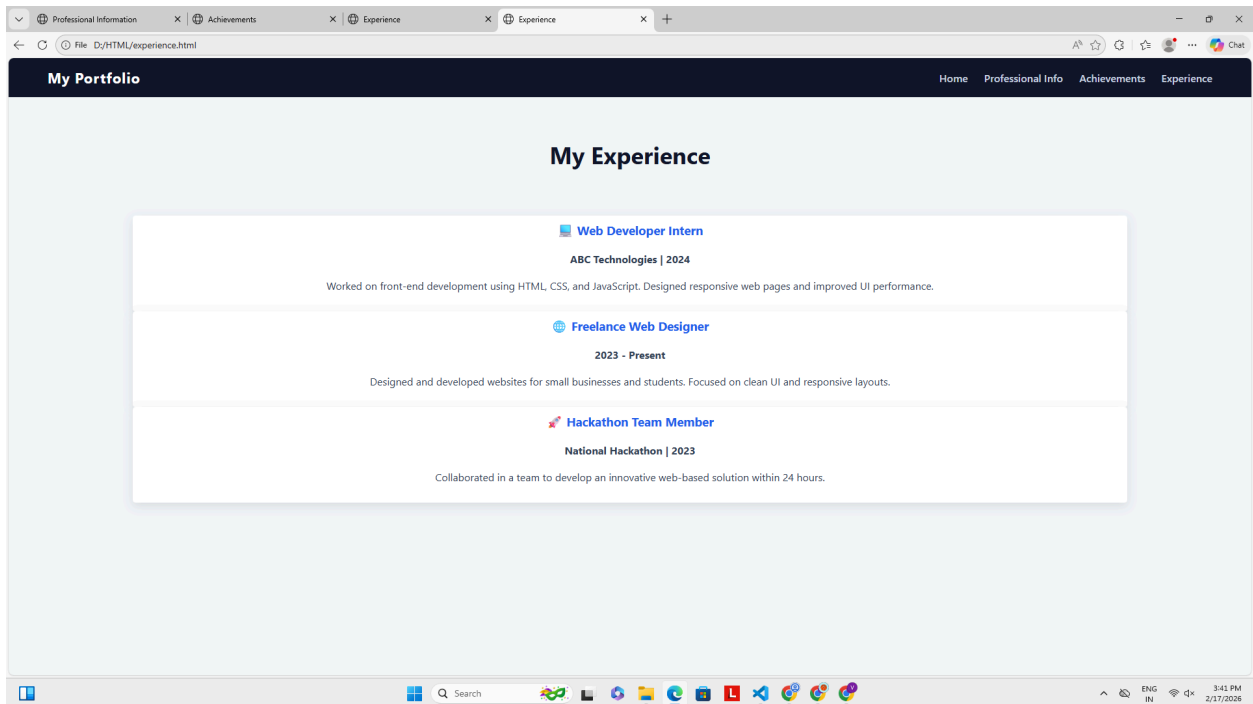
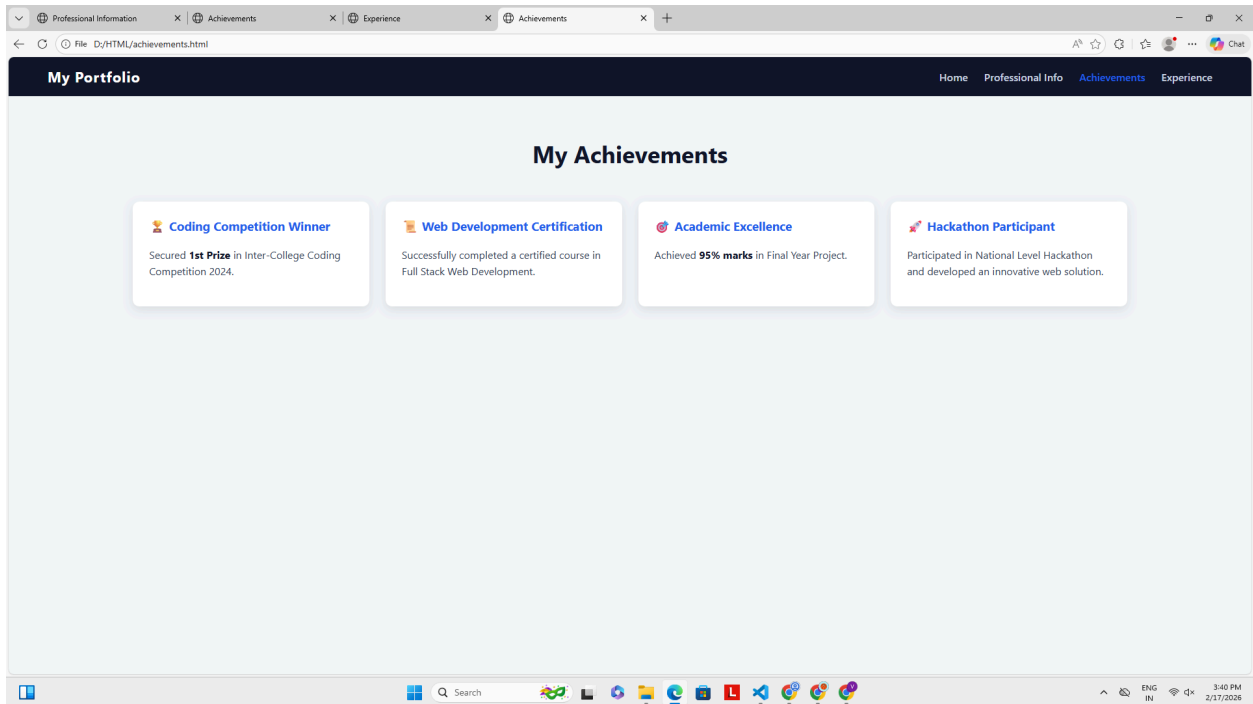
```

<center>
    <div class="content">
        <h3>💻 Web Developer Intern</h3>
        <p><strong>ABC Technologies |
2024</strong></p>
        <p>Worked on front-end
development using HTML, CSS, and
JavaScript. Designed responsive web pages
and improved UI performance.</p>
    </div>
    <div class="content">
        <h3>🌐 Freelance Web
Designer</h3>
        <p><strong>2023 -
Present</strong></p>
        <p>Designed and developed
websites for small businesses and students.
Focused on clean UI and responsive
layouts.</p>
    </div>
    <div class="content">
        <h3>🚀 Hackathon Team
Member</h3>
        <p><strong>National Hackathon |
2023</strong></p>
        <p>Collaborated in a team to
develop an innovative web-based solution
within 24 hours.</p>
    </div>
</center>
</div>
</body>
</html>

```

Output:





2. Smart Shopping Cart Application using React.js Components

Program Coding:

App.jsx:

```
import { useState } from "react"; //State
import ProductCard from
"./ProductCard.jsx";
import Cart from "./Cart.jsx";
import "./App.css";

// Functional Components
function App() {
  const products = [
    { id: 1, name: "Headphones", price: 1500
  },
    { id: 2, name: "Smart Watch", price: 2500
  },
    { id: 3, name: "Bluetooth Speaker", price:
3000 }
  ];

  const [cart, setCart] = useState([]); //State

  const addToCart = (product) => {
    setCart([...cart, product]);
  };

  const removeFromCart =
(indexToRemove) => {
    setCart(cart.filter((_, index) => index !==
indexToRemove));
  };

  return (
    <div className="container">
      <h1>🛒 Smart Shopping Cart</h1>

      <div className="layout">
```

```
<div>
  <h2>Products</h2>
  <div className="product-list">
    {products.map((product) => (
      //Props
      <ProductCard
        key={product.id}
        product={product}
        addToCart={addToCart}
      />
    ))}
  </div>
  <div>
    <Cart cart={cart}
      removeFromCart={removeFromCart} />
    //Props
  </div>
</div>
);
}

export default App;

Index.jsx:
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";
import "./index.css";

const root =
ReactDOM.createRoot(document.getElemen
tById("root"));
root.render(<App />);
```

ProductCard.jsx:

//Functional Components

```
function ProductCard({ product, addToCart
}) {
  return (
    <div className="product-card">
      <h3>{product.name}</h3>
      <p
className="price">₹ {product.price}</p>
      <button onClick={() =>
addToCart(product)}>
        Add to Cart
      </button>
    </div>
  );
}

export default ProductCard;
```

Cart.jsx:

//Functional Components

```
function Cart({ cart, removeFromCart }) {
  const total = cart.reduce((sum, item) =>
sum + item.price, 0);

  return (
    <div className="cart">
      <h2>📋 Cart Summary</h2>

      {cart.length === 0 ? (
        <p className="empty">Your cart is
empty</p>
      ) : (
        <ul>
          {cart.map((item, index) => (
            <li key={index}>
              <span>
                {item.name} – ₹ {item.price}
              </span>
            </li>
          )}
        </ul>
      )}
    </div>
  );
}
```

```
      <button
        className="remove"
        onClick={() =>
removeFromCart(index)}
      >
        Remove
      </button>
    </li>
  ))}
</ul>

  <h3>Total: ₹ {total}</h3>
</div>
);
}

export default Cart;
```

App.css:

```
body {
  margin: 0;
  background-color: #f4f6f8;
}

/* OUTER BORDER */
.container {
  font-family: Arial, sans-serif;
  width: 900px;
  margin: 40px auto;
  padding: 20px;
  background-color: white;
  border: 2px solid #333;
  border-radius: 10px;
}

h1 {
```

```

    text-align: center;
    margin-bottom: 20px;
}

.layout {
    display: flex;
    gap: 20px;
}

/* PRODUCTS */
.product-list {
    display: flex;
    gap: 15px;
}

.product-card {
    border: 1px solid #ccc;
    border-radius: 6px;
    padding: 15px;
    width: 180px;
    text-align: center;
}

.product-card .price {
    font-weight: bold;
}

.product-card button {
    background-color: #4caf50;
    color: white;
    border: none;
    padding: 8px;
    margin-top: 10px;
    cursor: pointer;

```

```

    border-radius: 4px;
}

/* CART */
.cart {
    border: 1px solid #ccc;
    border-radius: 6px;
    padding: 15px;
    width: 280px;
}

.cart ul {
    list-style: none;
    padding: 0;
}

.cart li {
    display: flex;
    justify-content: space-between;
    margin-bottom: 10px;
}

.remove {
    background-color: #e53935;
    color: white;
    border: none;
    padding: 4px 8px;
    cursor: pointer;
    border-radius: 4px;
}

.empty {
    color: gray;
}

```

Output:



Smart Shopping Cart

Products

Headphones
₹1500
[Add to Cart](#)

Smart Watch
₹2500
[Add to Cart](#)

Bluetooth Speaker
₹3000
[Add to Cart](#)

Cart Summary

Your cart is empty



Smart Shopping Cart

Products

Headphones
₹1500
[Add to Cart](#)

Smart Watch
₹2500
[Add to Cart](#)

Bluetooth Speaker
₹3000
[Add to Cart](#)

Cart Summary

Headphones – ₹1500 [Remove](#)

Smart Watch – ₹2500 [Remove](#)

Bluetooth Speaker – ₹3000 [Remove](#)

Total: ₹7000



Smart Shopping Cart

Products

Headphones
₹1500
[Add to Cart](#)

Smart Watch
₹2500
[Add to Cart](#)

Bluetooth Speaker
₹3000
[Add to Cart](#)

Cart Summary

Headphones – ₹1500 [Remove](#)

Total: ₹1500

13

3. Internship Application Form by Implementing Event Handling and Form Validation in React.js

Program Coding:

App.jsx :

```
import InternshipApplication from
"./Components/InternshipApplication";
```

```
function App() {
  return <InternshipApplication />;
}
```

```
export default App;
```

InternshipApplication.jsx :

```
import React, { useState } from "react";
```

```
function InternshipApplication() {
  const [formData, setFormData] =
  useState({
    fullName: "",
    email: "",
    mobile: "",
    college: "",
    cgpa: "",
    domain: "",
    resume: null,
    agree: false
  });

  const [errors, setErrors] = useState({});
  const [submitted, setSubmitted] =
  useState(false);

  const handleChange = (e) => {
    const { name, value, type, checked, files }
    = e.target;
```

```
    setFormData({
      ...formData,
      [name]:
        type === "checkbox"
          ? checked
          : type === "file"
            ? files[0]
            : value
    });
  };

  const validate = () => {
    let newErrors = {};

    if (!formData.fullName ||
    formData.fullName.length < 3)
      newErrors.fullName = "Full Name must
      be at least 3 characters";

    if (!/^S+@\S+\.\S+/.test(formData.email))
      newErrors.email = "Invalid email
      format";

    if (!/^d{10}$/.test(formData.mobile))
      newErrors.mobile = "Mobile number
      must be 10 digits";

    if (!formData.college)
      newErrors.college = "College name
      required";

    if (formData.cgpa < 0 || formData.cgpa >
    10)
      newErrors.cgpa = "CGPA must be
      between 0 and 10";
```

```

    if (!formData.domain)
      newErrors.domain = "Select internship
domain";

    if (!formData.resume)
      newErrors.resume = "Please upload your
resume";

    if (!formData.agree)
      newErrors.agree = "You must accept
terms";

    return newErrors;
  };

  const handleSubmit = (e) => {
    e.preventDefault();
    const validationErrors = validate();
    setErrors(validationErrors);

    if (Object.keys(validationErrors).length
=== 0) {
      setSubmitted(true);
      alert("🎉 Application Submitted
Successfully!");
    }
  };

  return (
    <div style={styles.page}>
      <div style={styles.card}>
        <h2 style={styles.heading}>Internship
Application Form</h2>

        <form onSubmit={handleSubmit}>

          <input style={styles.input}
name="fullName" placeholder="Full Name"
onChange={handleChange} />

```

```

      <p
style={styles.error}>{errors.fullName}</p>

      <input style={styles.input}
name="email" placeholder="Email"
onChange={handleChange} />
      <p
style={styles.error}>{errors.email}</p>

      <input style={styles.input}
name="mobile" placeholder="Mobile
Number" onChange={handleChange} />
      <p
style={styles.error}>{errors.mobile}</p>

      <input style={styles.input}
name="college" placeholder="College
Name" onChange={handleChange} />
      <p
style={styles.error}>{errors.college}</p>

      <input style={styles.input}
type="number" name="cgpa"
placeholder="CGPA (0-10)"
onChange={handleChange} />
      <p
style={styles.error}>{errors.cgpa}</p>

      <select style={styles.input}
name="domain"
onChange={handleChange}>
        <option value="">Select Internship
Domain</option>
        <option value="Web">Web
Development</option>
        <option value="Data Science">Data
Science</option>
        <option value="AI">Artificial
Intelligence</option>
        <option value="Cyber
Security">Cyber Security</option>

```

```

        <option value="Cloud
Computing">Cloud Computing</option>
        <option value="Mobile
App">Mobile App Development</option>
        <option value="UI/UX">UI/UX
Design</option>
        <option
value="DevOps">DevOps</option>
    </select>
    <p
style={styles.error}>{errors.domain}</p>

    <label style={styles.label}>Upload
Resume (PDF Only)</label>
    <input style={styles.input}
type="file" name="resume"
onChange={handleChange} />
    <p
style={styles.error}>{errors.resume}</p>

    <label style={styles.checkboxLabel}>
    <input type="checkbox"
name="agree" onChange={handleChange}
/>
        I agree to the terms & conditions
    </label>
    <p
style={styles.error}>{errors.agree}</p>

    <button style={styles.button}
type="submit">
        Apply Now
    </button>
</form>

    {/* Optional: You can remove this
since alert is shown */}
    {submitted && (
        <p style={styles.success}>
            🎉 Application Submitted
            Successfully!

```

```

        </p>
    )}
</div>
</div>
);
}

const styles = {
  page: {
    backgroundColor: "#f4f6f9",
    minHeight: "100vh",
    display: "flex",
    justifyContent: "center",
    alignItems: "center"
  },
  card: {
    backgroundColor: "#ffffff",
    padding: "30px",
    borderRadius: "10px",
    boxShadow: "0 4px 12px rgba(0,0,0,0.1)",
    width: "400px"
  },
  heading: {
    textAlign: "center",
    color: "#2c3e50",
    marginBottom: "20px"
  },
  input: {
    width: "100%",
    padding: "8px",
    marginTop: "8px",
    borderRadius: "5px",
    border: "1px solid #ccc"
  },
  label: {
    marginTop: "10px",
    fontWeight: "bold",
    display: "block"
  },
  checkboxLabel: {
    marginTop: "10px",

```



```
    display: "block"
  },
  button: {
    width: "100%",
    padding: "10px",
    marginTop: "15px",
    backgroundColor: "#3498db",
    color: "white",
    border: "none",
    borderRadius: "5px",
    cursor: "pointer",
    fontWeight: "bold"
  },
  error: {
```

```
    color: "red",
    fontSize: "12px",
    margin: "4px 0"
  },
  success: {
    color: "green",
    textAlign: "center",
    marginTop: "15px",
    fontWeight: "bold"
  }
};

export default InternshipApplication;
```

Output:

Internship Application Form

Upload Resume (PDF Only)

No file chosen

☐ I agree to the terms & conditions

Internship Application Form

Upload Resume (PDF Only)

No file chosen

Please upload your resume

☒ I agree to the terms & conditions

← → ↻ 📄 localhost:5173

localhost:5173 says

📄 Application Submitted Successfully!

OK

Internship Application Form

ramsree

ramsree2905@gmail.com

7598388000

PSGR Krishnammal college

9

Web Development

Upload Resume (PDF Only)

Choose File

 resume.pdf

☒ I agree to the terms & conditions

Apply Now

4. Student Dashboard Using React Router with Multi-page Applications

Program Coding:

StudentDashboard.jsx

```
import { useState } from "react";
```

```
const StudentDashboard = () => {  
  const [page, setPage] =  
  useState("dashboard");
```

```
  const students = [  
    { id: 1, name: "Pavi", dept: "CSE", year:  
      "III" },  
    { id: 2, name: "Nisha", dept: "IT", year:  
      "II" },  
    { id: 3, name: "Swathi", dept: "ECE",  
      year: "I" },  
  ];
```

```
  const renderContent = () => {  
    if (page === "dashboard") {  
      return (  
        <div>  
          <h2> Dashboard</h2>  
          <p>Total Students:  
<strong>{students.length}</strong></p>  
        </div>  
      );  
    }  
  }
```

```
  if (page === "students") {  
    return (  
      <div>  
        <h2> Student List</h2>  
        <table style={styles.table}>  
          <thead>  
            <tr>  
              <th style={styles.th}>Name</th>
```

```
              <th  
style={styles.th}>Department</th>  
              <th style={styles.th}>Year</th>  
            </tr>  
          </thead>  
          <tbody>  
            {students.map((s) => (  
              <tr key={s.id}>  
                <td  
style={styles.td}>{s.name}</td>  
                <td  
style={styles.td}>{s.dept}</td>  
                <td  
style={styles.td}>{s.year}</td>  
              </tr>  
            ))}  
          </tbody>  
        </table>  
      </div>  
    );  
  }
```

```
  if (page === "profile") {  
    return (  
      <div>  
        <h2> Admin Profile</h2>  
        <p><strong>Name:</strong>  
Admin</p>  
        <p><strong>Role:</strong> Student  
Manager</p>  
      </div>  
    );  
  }  
};
```

```
  return (  
    <div style={styles.container}>
```

```

    { /* Navigation */
    <div style={styles.sidebar}>
      <h3>🎓 Student Panel</h3>
      <button style={styles.navBtn}
onClick={() => setPage("dashboard")}>
        Dashboard
      </button>
      <button style={styles.navBtn}
onClick={() => setPage("students")}>
        Students
      </button>
      <button style={styles.navBtn}
onClick={() => setPage("profile")}>
        Profile
      </button>
    </div>

    { /* Content Area */
    <div
style={styles.content}>{renderContent()}</div>
    </div>
  );
};

const styles = {
  container: {
    minHeight: "100vh",
    display: "flex",
    fontFamily: "Arial, sans-serif",
    backgroundColor: "#f4f6ff",
  },
  sidebar: {
    width: "220px",
    backgroundColor: "#6c5ce7",
    color: "white",
    padding: "20px",
    display: "flex",
    flexDirection: "column",
    gap: "10px",
  },

```

```

    navBtn: {
      backgroundColor: "white",
      color: "#6c5ce7",
      border: "none",
      padding: "8px",
      borderRadius: "5px",
      cursor: "pointer",
      textAlign: "left",
    },
    content: {
      flex: 1,
      padding: "30px",
      backgroundColor: "#f4f6ff",
    },
    table: {
      width: "100%",
      borderCollapse: "collapse",
      marginTop: "15px",
      backgroundColor: "white",
    },
    th: {
      border: "1px solid #ccc",
      padding: "10px",
      backgroundColor: "#dfe6e9",
    },
    td: {
      border: "1px solid #ccc",
      padding: "10px",
      textAlign: "center",
    },
  };

```

export default StudentDashboard;

App.jsx:

```

import StudentDashboard from
"./FullStack/React/StudentDashboard"
function App() {
  return<StudentDashboard/>
}
export default Ap

```

Output:

 **Student Panel**

Dashboard

Students

Profile

 **Dashboard**

Total Students: **3**

 **Student Panel**

Dashboard

Students

Profile

 **Student List**

Name	Department	Year
Pavi	CSE	III
Nisha	IT	II
Swathi	ECE	I

 **Student Panel**

Dashboard

Students

Profile

 **Admin Profile**

Name: Admin

Role: Student Manager

5. Smart Task Tracker Application Using React and Redux

Terminal:

```
npm install redux react-redux
@reduxjs/toolkit
```

Program Coding:

store.js:

```
import { createStore, applyMiddleware }
from "redux";
import { taskReducer } from
"./reducer/taskReducer.js";
import logger from
"./middleware/logger";

const store = createStore(taskReducer,
applyMiddleware(logger));

export default store;
```

actions/taskActions.js

```
export const addTask = (text) => {
  return {
    type: "ADD_TASK",
    payload: text,
  };
};

export const deleteTask = (id) => {
  return {
    type: "DELETE_TASK",
    payload: id,
  };
};

export const toggleTask = (id) => {
  return {
    type: "TOGGLE_TASK",
    payload: id,
  };
};
```

```
};
```

reducers/taskReducer.js

```
const initialState = {
  tasks: [],
};

export const taskReducer = (state =
initialState, action) => {
  switch (action.type) {
    case "ADD_TASK":
      return {
        ...state,
        tasks: [
          ...state.tasks,
          { id: Date.now(), text: action.payload,
            completed: false },
        ],
      };

    case "DELETE_TASK":
      return {
        ...state,
        tasks: state.tasks.filter(task => task.id
!== action.payload),
      };

    case "TOGGLE_TASK":
      return {
        ...state,
        tasks: state.tasks.map(task =>
task.id === action.payload
? { ...task, completed:
!task.completed }
: task
      ),
      };
  }
};
```

```

    default:
      return state;
    }
  };

```

middleware/logger.js

```

const logger = store => next => action => {
  console.log("Dispatching:", action);
  let result = next(action);
  console.log("Next State:", store.getState());
  return result;
};

```

```

export default logger;

```

index.js(vite means main.jsx)

```

import React from "react";
import ReactDOM from "react-dom/client";
import { Provider } from "react-redux";
import App from "../App";
import store from "../store";

```

```

const root =
ReactDOM.createRoot(document.getElementBy
  tById("root"));

```

```

root.render(
  <Provider store={store}>
    <App />
  </Provider>
);

```

components/AddTask.js:

```

import React, { useState } from "react";
import { useDispatch } from "react-redux";
import { addTask } from
  "../Actions/taskActions";

```

```

function AddTask() {
  const [text, setText] = useState("");
  const dispatch = useDispatch();

```

```

  const handleAdd = () => {
    if (text.trim() === "") return;
    dispatch(addTask(text));
    setText("");
  };

```

```

  return (
    <div
      style={{
        display: "flex",
        gap: "10px",
        marginBottom: "25px"
      }}
    >
      <input
        type="text"
        placeholder="Enter your task..."
        value={text}
        onChange={(e) =>
          setText(e.target.value)}
        style={{
          flex: 1,
          padding: "12px",
          borderRadius: "10px",
          border: "none",
          outline: "none",
          fontSize: "16px"
        }}
      />

```

```

      <button
        onClick={handleAdd}
        style={{
          padding: "12px 20px",
          borderRadius: "10px",
          border: "none",

```



```

        background: "linear-gradient(45deg,
#ff6b6b, #ff9f43)",
        color: "#fff",
        fontWeight: "bold",
        cursor: "pointer",
        transition: "0.3s"
      }}
    >
      Add
    </button>
  </div>
);
}

```

export default AddTask;

components/TaskList.js:

```

import React from "react";
import { useSelector, useDispatch } from
"react-redux";
import { deleteTask, toggleTask } from
"./Actions/taskActions";

function TaskList() {
  const tasks = useSelector((state) =>
state.tasks);
  const dispatch = useDispatch();

  return (
    <div
      style={{
        maxWidth: "600px",
        margin: "30px auto",
        padding: "20px",

        backgroundColor: "rgba(255,255,255,0.2)",
        color: "#fff",

        borderRadius: "12px",

```

```

        boxShadow: "0 4px 12px
rgba(0,0,0,0.08)",
        fontFamily: "Segoe UI, sans-serif"
      }}
    >
      <h2
        style={{
          textAlign: "center",
          marginBottom: "20px",
          color: "#333"
        }}
      >
        Task List
      </h2>

      {tasks.length === 0 ? (
        <p style={{ textAlign: "center", color:
"#777" }}>
          No tasks available
        </p>
      ) : (
        tasks.map((task) => (
          <div
            key={task.id}
            style={{
              display: "flex",
              justifyContent: "space-between",
              alignItems: "center",
              padding: "12px",
              marginBottom: "10px",
              backgroundColor: "#f9fafc",
              borderRadius: "8px",
              border: "1px solid #e5e7eb"
            }}
          >
            <span
              onClick={() =>
dispatch(toggleTask(task.id))}
              style={{
                textDecoration: task.completed ?
"line-through" : "none",

```

```

        cursor: "pointer",
        color: task.completed ? "#9ca3af"
: "#111827",
        fontWeight: "500"
    }}
    >
    {task.text}
  </span>
  <button
    onClick={() =>
dispatch(deleteTask(task.id))}
    style={{
      padding: "6px 12px",
      backgroundColor: "#ef4444",
      color: "#fff",
      border: "none",
      borderRadius: "6px",
      cursor: "pointer",
      fontSize: "14px"
    }}
  >
    Delete
  </button>
</div>
))
})
</div>
);
}

```

export default TaskList;

app.js:

```

import React from "react";
import AddTask from
"./components/AddTask";
import TaskList from
"./components/TaskList";

```

```

function App() {
  return (

```

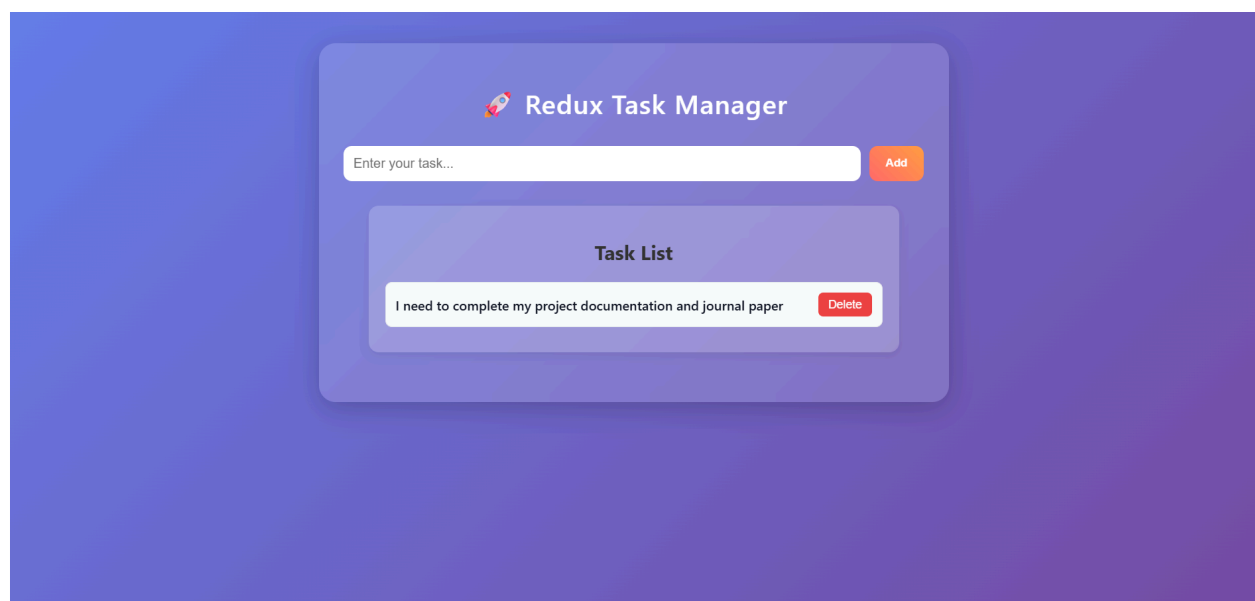
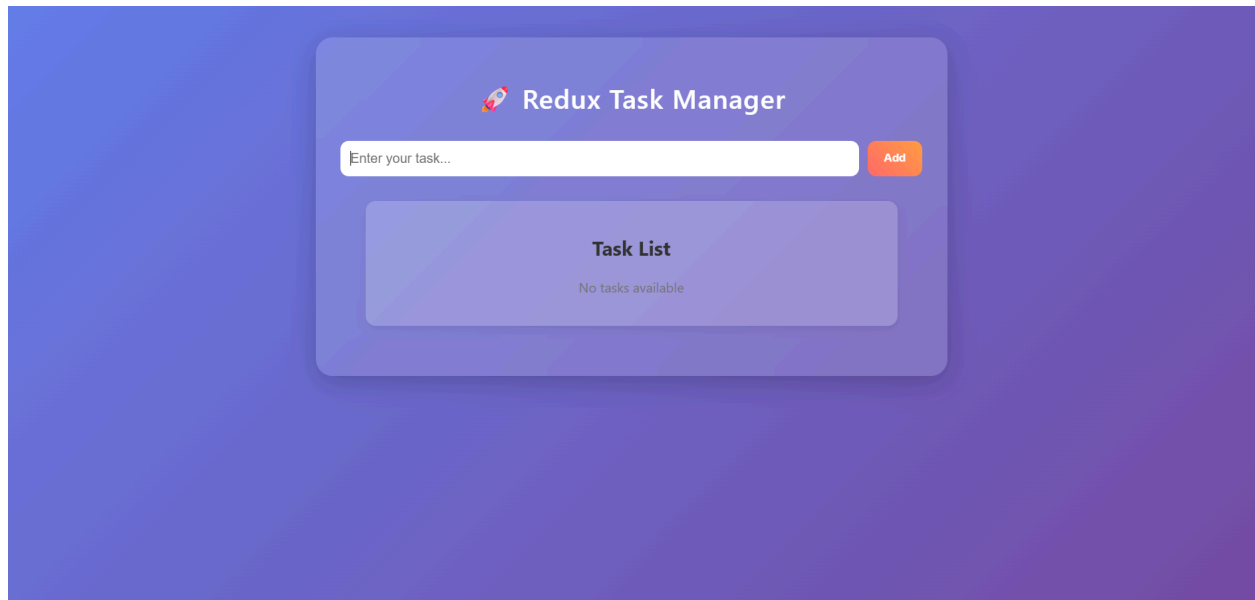
```

<div
  style={{
    minHeight: "100vh",
    background: "linear-gradient(135deg,
#667eea, #764ba2)",
    padding: "40px 20px",
    fontFamily: "Segoe UI, sans-serif"
  }}
>
  <div
    style={{
      maxWidth: "700px",
      margin: "auto",
      background: "rgba(255, 255, 255,
0.15)",
      backdropFilter: "blur(15px)",
      padding: "30px",
      borderRadius: "20px",
      boxShadow: "0 8px 32px
rgba(0,0,0,0.2)",
      color: "#fff"
    }}
  >
    <h1
      style={{
        textAlign: "center",
        marginBottom: "30px",
        fontWeight: "600",
        letterSpacing: "1px"
      }}
    >
      🚀 Redux Task Manager
    </h1>

    <AddTask />
    <TaskList />
  </div>
</div>
);
}
export default App;

```

Output:



6. Online Job Application Form Using Angular

Program Coding:

app.config.ts

```
import { ApplicationConfig } from
 '@angular/core';
import { provideRouter } from
 '@angular/router';
import { routes } from './app.routes';

export const appConfig: ApplicationConfig
= {
  providers: [provideRouter(routes)]
};
```

app.routes.ts

```
import { Routes } from '@angular/router';
import { App } from './app';

export const routes: Routes =
  { path: "", component: App }
];
```

app.html

```
<div class="container">
  <h2>Online Job Application</h2>
  <p class="mandatory-note">
    <span class="star">*</span> indicates
    mandatory fields
  </p>
  <form #jobForm="ngForm"
    (ngSubmit)="submitForm(jobForm)">
    <!-- Personal Details -->
    <h3>Personal Details</h3>
    <label>Full Name <span
    class="star">*</span></label>
    <input
```

```
    type="text"
    name="name"
    [(ngModel)]="applicant.name"
    required
    #name="ngModel"
    [class.invalid]="name.invalid &&
    name.touched">
    <div class="error" *ngIf="name.invalid &&
    name.touched">
      Name is required
    </div>
    <label>Email <span
    class="star">*</span></label>
    <input
    type="text"
    name="email"
    [(ngModel)]="applicant.email"
    required
    pattern="^[a-zA-Z0-9._%+~]+@[a-zA-Z0-9.
    -]+\.[a-zA-Z]{2,}$"
    #email="ngModel"
    [class.invalid]="email.invalid &&
    email.touched">
    <div class="error"
    *ngIf="email.errors?.['required'] &&
    email.touched">
      Email is required
    </div>
    <div class="error"
    *ngIf="email.errors?.['pattern'] &&
    email.touched">
      Enter a valid email address (example:
      name123@gmail.com)
    </div>
    <label>Phone Number <span
    class="star">*</span></label>
    <input
    type="text"
```

```

name="phone"
[(ngModel)]="applicant.phone"
pattern="[0-9]{10}"
required
#phone="ngModel"
[class.invalid]="phone.invalid &&
phone.touched">
<div class="error" *ngIf="phone.invalid
&& phone.touched">
  Enter valid 10-digit phone number
</div>
<label>Address <span
class="star">*</span></label>
<textarea name="address"
[(ngModel)]="applicant.address"
required></textarea>
<!-- Job Details -->
<h3>Job Details</h3>
<label>Position Applying For <span
class="star">*</span></label>
<select name="position"
[(ngModel)]="applicant.position" required>
  <option value="">Select</option>
  <option *ngFor="let p of
positions">{{p}}</option>
</select>
<label>Preferred Location <span
class="star">*</span></label>
<select name="location"
[(ngModel)]="applicant.location" required>
  <option value="">Select</option>
  <option *ngFor="let l of
locations">{{l}}</option>
</select>
<label>Expected Salary <span
class="star">*</span></label>
<input type="number" name="salary"
[(ngModel)]="applicant.salary" required>
<!-- Education -->
<h3>Educational Qualification</h3>

```

```

<label>Highest Qualification <span
class="star">*</span></label>
<select name="qualification"
[(ngModel)]="applicant.qualification"
required>
  <option value="">Select</option>
  <option *ngFor="let q of
qualifications">{{q}}</option>
</select>
<label>University / College <span
class="star">*</span></label>
<select name="university"
[(ngModel)]="applicant.university"
required>
  <option value="">Select</option>
  <option *ngFor="let u of
universities">{{u}}</option>
</select>
<label>Year of Passing <span
class="star">*</span></label>
<select name="year"
[(ngModel)]="applicant.year" required>
  <option value="">Select</option>
  <option *ngFor="let y of
years">{{y}}</option>
</select>
<!-- Skills -->
<h3>Skills <span
class="star">*</span></h3>
<div class="checkbox-group" *ngFor="let
skill of skillsList">
  <label>
    <input
      type="checkbox"
      [value]="skill"
      (change)="onSkillChange($event)">
    {{skill}}
  </label>
</div>
<!-- Experience -->

```

```

    <h3>Experience <span
class="star">*</span></h3>
<label class="radio-group">
    <input type="radio" name="exp"
value="Fresher"
[(ngModel)]="applicant.exp">
    Fresher
</label>
<label class="radio-group">
    <input type="radio" name="exp"
value="Experienced"
[(ngModel)]="applicant.exp">
    Experienced
</label>
<div *ngIf="applicant.exp ===
'Experienced'">
    <label>Years of Experience</label>
    <input type="number" name="yearsExp"
[(ngModel)]="applicant.yearsExp">
</div>
    <!-- Resume Upload -->
    <h3>Upload Resume <span
class="star">*</span></h3>
    <input type="file">
    <!-- Declaration -->
    <div class="agree">
    <input
type="checkbox"
name="agree"
[(ngModel)]="applicant.agree"
required
#agree="ngModel" >
    <span>
    I confirm the above information is correct
    <span class="star">*</span>
    </span>
</div>
<div class="error" *ngIf="agree.invalid &&
agree.touched">
    You must accept before submitting
</div>

```

```

    <button type="submit"
[disabled]="jobForm.invalid">
    Submit Application
    </button>
</form>
</div>

```

app.css

```

.container {
    width: 500px;
    margin: 30px auto;
    background: white;
    padding: 25px;
    border-radius: 8px;
    box-shadow: 0 0 10px rgba(0,0,0,0.1);
    font-family: Arial;
}
h2 {
    text-align: center;
}
h3 {
    margin-top: 20px;
    color: #007BFF;
}
input, select, textarea {
    width: 100%;
    padding: 8px;
    margin: 8px 0 15px 0;
}
button {
    width: 100%;
    padding: 12px;
    background: #007BFF;
    color: white;
    border: none;
}
button:disabled {
    background: gray;
}
.checkbox-group label,
.radio-group {

```

```

display: flex;
align-items: center;
gap: 8px;
margin-bottom: 6px;
}
.checkbox-group input,
.radio-group input {
width: auto;
}
.star {
color: red;
font-weight: bold;
}
.mandatory-note {
margin-bottom: 15px;
font-size: 14px;
}
.error {
color: red;
font-size: 12px;
margin-top: -10px;
margin-bottom: 10px;
}
.invalid {
border: 2px solid red;
}
.agree {
display: flex;
align-items: flex-start;
gap: 8px;
margin-top: 10px;
}
.agree input {
margin-top: 3px;
width: auto;
}

```

OUTPUT:

This screenshot shows the first part of an online job application form. The form is titled "Online Job Application" and includes a note that an asterisk (*) indicates mandatory fields. It is divided into two sections: "Personal Details" and "Job Details".

Personal Details

- Full Name *
- Email *
- Phone Number *
- Address *

Job Details

- Position Applying For *
- Preferred Location *
- Expected Salary *

The form is displayed in a web browser window with the address bar showing "localhost:4200". The Windows taskbar at the bottom shows the date and time as 10:58 am on 18/02/2026.

This screenshot shows the second part of the online job application form. It includes sections for "Educational Qualification", "Skills", "Experience", and "Upload Resume".

Educational Qualification

- Highest Qualification *
- University / College *
- Year of Passing *

Skills

- ☐ Java
- ☐ Python
- ☐ HTML
- ☐ CSS
- ☐ JavaScript

Experience

- ☐ Fresher
- ☐ Experienced

Upload Resume

- Choose File | No file chosen
- ☐ I confirm the above information is correct *
- Submit Application

The form is displayed in a web browser window with the address bar showing "localhost:4200". The Windows taskbar at the bottom shows the date and time as 11:01 am on 18/02/2026.

Myapp

localhost:4200

Online Job Application

* indicates mandatory fields

Personal Details

Full Name *
Sri Pechi Pavithra H

Email *
23bcs112@psgkwc.ac.in

Phone Number *
9790489954

Address *
abc

Job Details

Position Applying For *
Software Developer

Preferred Location *
Bangalore

Expected Salary *
25000

Educational Qualification

Myapp

localhost:4200

Highest Qualification *
UG

University / College *
Anna University

Year of Passing *
2025

Skills *

☒ Java
☐ Python
☐ HTML
☐ CSS
☐ JavaScript

Experience *

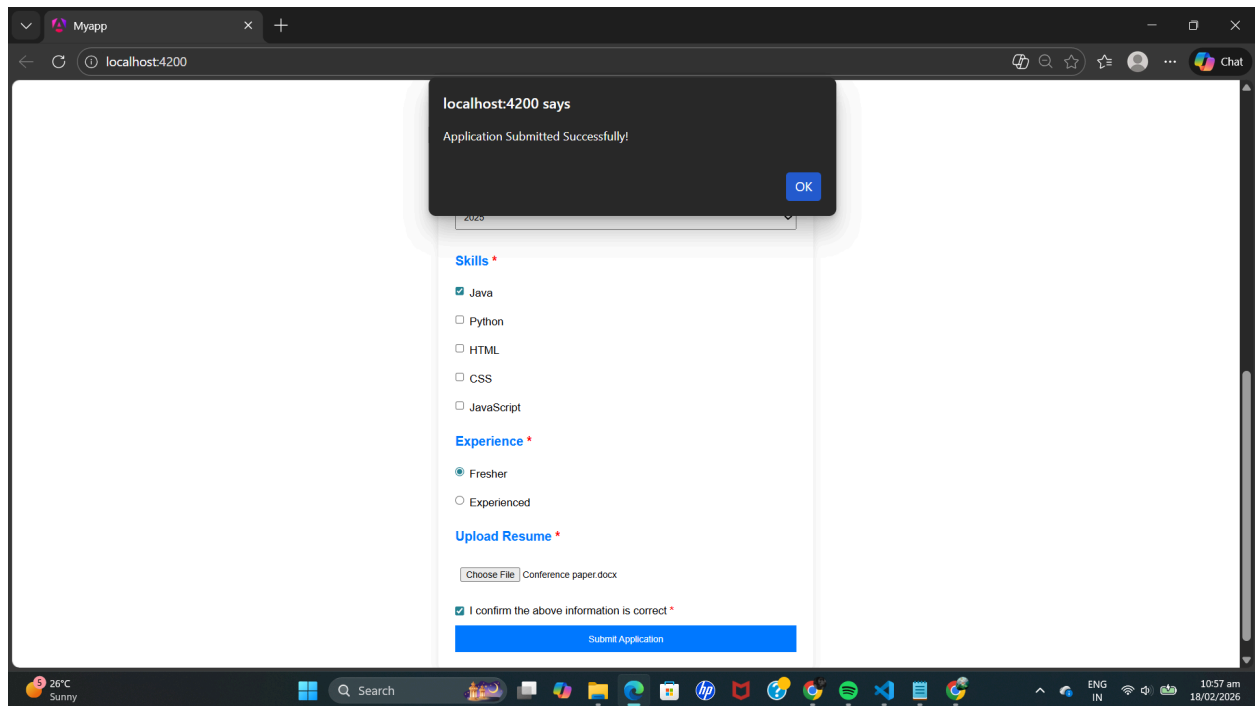
☒ Fresher
☐ Experienced

Upload Resume *

Choose File Conference paper.docx

☒ I confirm the above information is correct *

Submit Application



7. Hospital Management System for Patients using AngularJS Services and REST API

Program Coding:

Index.html:

```
<!DOCTYPE html>
<html ng-app="app">
<head>
  <title>Hospital Management
System</title>
  <script
src="https://ajax.googleapis.com/ajax/libs/a
ngularjs/1.8.2/angular.min.js"></script>

  <style>
    body {
      font-family: Arial, Helvetica,
sans-serif;
      background: linear-gradient(to right,
#4e73df, #1cc88a);
      margin: 0;
      padding: 0;
    }

    .container {
      width: 60%;
      margin: 50px auto;
      background: white;
      padding: 30px;
      border-radius: 10px;
      box-shadow: 0 5px 15px
rgba(0,0,0,0.2);
    }

    h1 {
      text-align: center;
      color: #4e73df;
```

```

  }
  h2 {
    margin-top: 30px;
    color: #333;
  }

  input {
    padding: 10px;
    margin: 5px 0;
    width: 30%;
    border-radius: 5px;
    border: 1px solid #ccc;
  }

  button {
    padding: 10px 20px;
    background-color: #4e73df;
    color: white;
    border: none;
    border-radius: 5px;
    cursor: pointer;
  }

  button:hover {
    background-color: #2e59d9;
  }

  table {
    width: 100%;
    margin-top: 15px;
    border-collapse: collapse;
  }

  table, th, td {
    border: 1px solid #ddd;
```

```

    }

    th {
        background-color: #4e73df;
        color: white;
        padding: 10px;
    }

    td {
        padding: 10px;
        text-align: center;
    }

    tr:nth-child(even) {
        background-color: #f2f2f2;
    }

</style>
</head>

<body ng-controller="ctrl">

<div class="container">
    <h1>Hospital Management System</h1>

    <h2>Add Patient</h2>

    <input ng-model="p.name"
placeholder="Name">
    <input ng-model="p.age"
placeholder="Age">
    <input ng-model="p.gender"
placeholder="Gender">
    <button ng-click="add()">Add
Patient</button>

    <h2>Patient List</h2>

    <table>
        <tr>
            <th>ID</th>

```

```

            <th>Name</th>
            <th>Age</th>
            <th>Gender</th>
        </tr>
        <tr ng-repeat="x in patients">
            <td>{{x.id}}</td>
            <td>{{x.name}}</td>
            <td>{{x.age}}</td>
            <td>{{x.gender}}</td>
        </tr>
    </table>
</div>

<script>
var app = angular.module("app", []);

app.controller("ctrl", function($scope, $http)
{

    function load(){

        $http.get("/patients").then(function(res){
            $scope.patients = res.data;
        });
    }

    $scope.add = function(){
        $http.post("/patients",
        $scope.p).then(function(){
            $scope.p = {};
            load();
        });
    };

    load();
});
</script>

</body>
</html>

```

Server.js

```
const express = require("express");  
const app = express();
```

```
app.use(express.json());  
app.use(express.static(__dirname));
```

```
let patients = [];  
let id = 1;
```

```
// Get all patients  
app.get("/patients", (req, res) => {  
  res.json(patients);  
});
```

```
// Add patient
```

```
app.post("/patients", (req, res) => {  
  const patient = {  
    id: id++,  
    name: req.body.name,  
    age: req.body.age,  
    gender: req.body.gender  
  };  
  patients.push(patient);  
  res.json(patient);  
});
```

```
app.listen(3000, () => {  
  console.log("Server running at  
http://localhost:3000");  
});
```

Output:

The screenshot shows a web browser window with the URL `localhost:3000`. The page has a blue and green gradient background. A white card in the center contains the title "Hospital Management System" in blue. Below the title, there is a section titled "Add Patient" with three input fields: "Name", "Age", and "Gender". A blue "Add Patient" button is located below these fields. Underneath, there is a section titled "Patient List" which contains an empty table with four columns: "ID", "Name", "Age", and "Gender".

ID	Name	Age	Gender
----	------	-----	--------

This screenshot shows the same web application after a patient has been added. The "Add Patient" form remains the same. The "Patient List" table now contains one row of data.

ID	Name	Age	Gender
1	swetha	20	female

8.Simple Attendance Tracking System using Node.js: Installation and HTTP Server Setup

Program Coding:

App.jsx:

```
import Attendance from "./Attendance";
function App() {
  return <Attendance/>;
}
export default App;
```

Attendance.jsx:

```
import { useState } from "react";
function Attendance() {
  const [students, setStudents] = useState([
    { name: "Varshini", present: false },
    { name: "sahana", present: false },
    { name: "rethanisha", present: false },
  ]);
  const toggleAttendance = (index) => {
    const updatedStudents = [...students];
    updatedStudents[index].present =
      !updatedStudents[index].present;
    setStudents(updatedStudents);
  };

  const presentCount = students.filter(s =>
    s.present).length;
  const absentCount = students.length -
    presentCount;

  return (
    <div className="app">
      <div className="card">
```

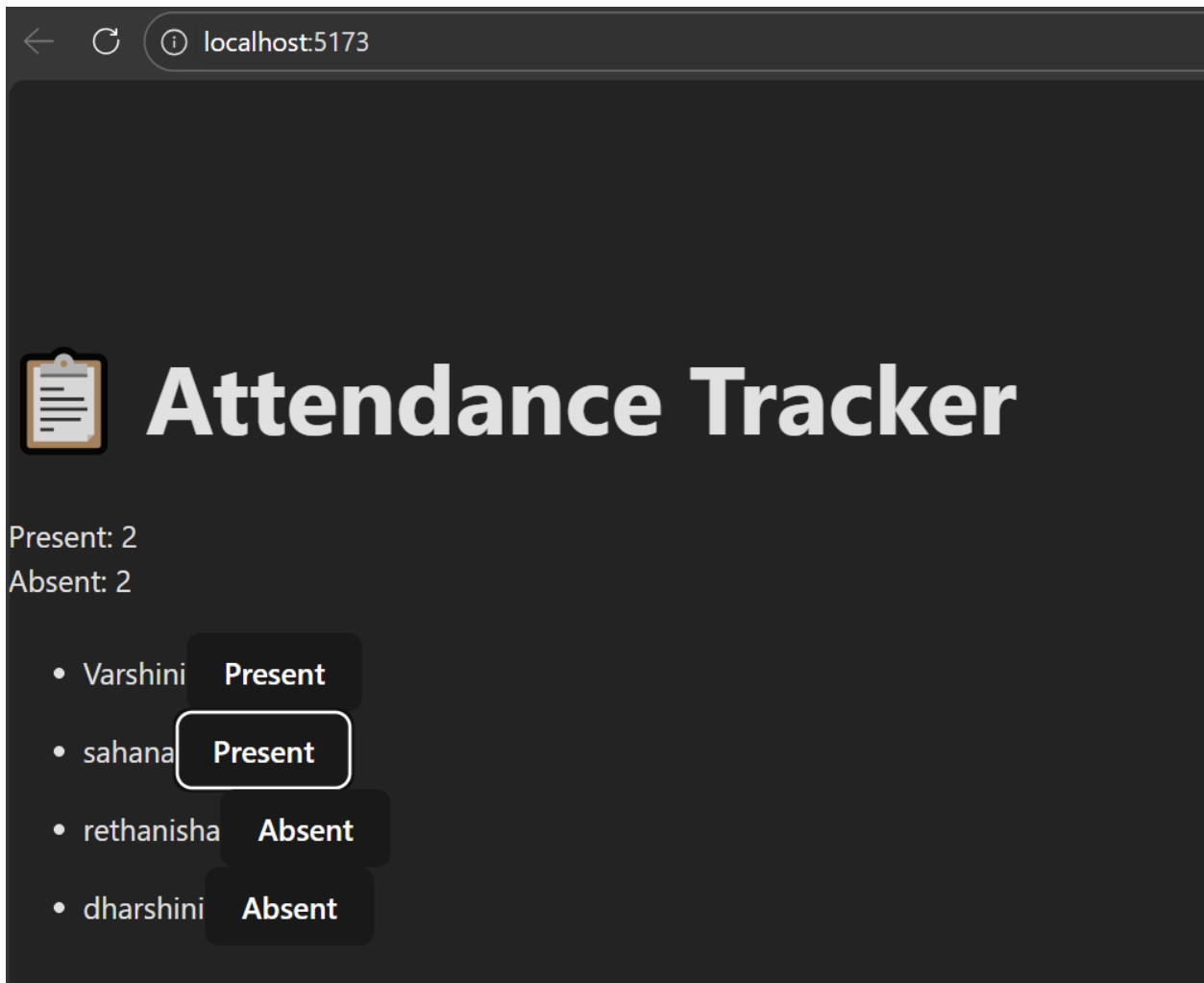
```
<h1> 📅 Attendance Tracker</h1>

    <div className="stats">
      <div className="present">Present:
        {presentCount}</div>
      <div className="absent">Absent:
        {absentCount}</div>
    </div>

    <ul>
      {students.map((student, index) => (
        <li key={index}>
          <span>{student.name}</span>
          <button
            className={student.present ?
              "btn-present" : "btn-absent"}
            onClick={() =>
              toggleAttendance(index)}
          >
            {student.present ? "Present" :
              "Absent"}
          </button>
        </li>
      ))}
    </ul>
  </div>
</div>

);
}
export default Attendance;
```

Output:



9. Hospital Appointment RESTful API using Express.js: routing, middleware and Error Handling

Program Coding:

index.js:

```
const express = require("express");
const appointmentRoutes =
  require("./routes/appointmentRoutes");
const logger =
  require("./middleware/logger");

const app = express();
const PORT = 3000;

// Middleware
app.use(express.json()); // parses JSON
app.use(logger); // logs requests

// Routes
app.use("/api", appointmentRoutes);

// Error handling
app.use((err, req, res, next) => {
  res.status(500).json({ error: err.message });
});

// Start server
app.listen(PORT, () => {
  console.log(`Hospital Appointment API
  running on port ${PORT}`);
});

//Middleware
logger.js:
function logger(req, res, next) {
  console.log(`${req.method} ${req.url}`);
  next();
}

module.exports = logger;
```

//ROUTING

appointmentRoutes.js:

```
const express = require("express");
const router = express.Router();

// Sample doctors
let doctors = [
  { id: 1, name: "Dr. Arun", specialization:
    "Cardiologist" },
  { id: 2, name: "Dr. Meena", specialization:
    "Dermatologist" }
];

// Pre-filled appointments
let appointments = [
  { id: 1, patientName: "Alice", doctorId: 1
  },
  { id: 2, patientName: "Bob", doctorId: 2 }
];

// GET doctors
router.get("/doctors", (req, res) =>
  res.json(doctors));

// GET appointments
router.get("/appointments", (req, res) =>
  res.json(appointments));

// POST appointment (optional, still works)
router.post("/appointments", (req, res) => {
  const { patientName, doctorId } =
    req.body;
  if (!patientName || !doctorId)
    return res.status(400).json({ error:
    "Patient name and doctor ID are required"
  });
});
```

```

    const appointment = { id:
appointments.length + 1, patientName,
doctorId };
    appointments.push(appointment);
    res.status(201).json({ message:
"Appointment booked successfully",
appointment });
});

// DELETE appointment (optional, still
works)
router.delete("/appointments/:id", (req, res)
=> {
    const id = parseInt(req.params.id);

```

```

    const initialLength = appointments.length;
    appointments = appointments.filter(app =>
app.id !== id);

    if (appointments.length === initialLength)
        return res.status(404).json({ error:
"Appointment not found" });

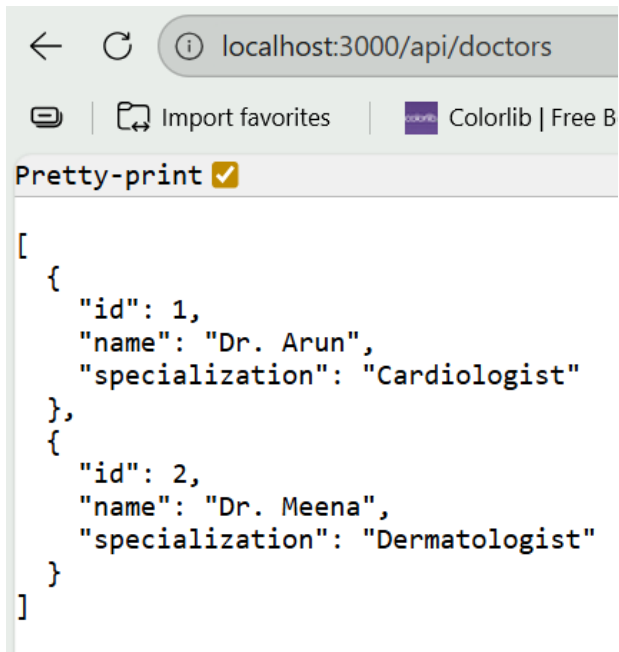
    res.json({ message: "Appointment
cancelled" });
});

module.exports = router;

```

Output:

GET Method: /api/doctors



A screenshot of a web browser window. The address bar shows 'localhost:3000/api/doctors'. Below the address bar, there are buttons for 'Import favorites' and 'Colorlib | Free B'. A 'Pretty-print' button with a checkmark is visible. The main content area displays a JSON array of two doctor objects. The first object has 'id': 1, 'name': 'Dr. Arun', and 'specialization': 'Cardiologist'. The second object has 'id': 2, 'name': 'Dr. Meena', and 'specialization': 'Dermatologist'.

```
[
  {
    "id": 1,
    "name": "Dr. Arun",
    "specialization": "Cardiologist"
  },
  {
    "id": 2,
    "name": "Dr. Meena",
    "specialization": "Dermatologist"
  }
]
```

GET Method: /api/appointments



A screenshot of a web browser window. The address bar shows 'localhost:3000/api/appointments'. Below the address bar, there are buttons for 'Import favorites' and 'Colorlib | Free Boo.'. A 'Pretty-print' button with a checkmark is visible. The main content area displays a JSON array of four appointment objects. Each object contains 'id', 'patientName', and 'doctorId'. The appointments are: 1. id: 1, patientName: 'Alice', doctorId: 1; 2. id: 2, patientName: 'Bob', doctorId: 2; 3. id: 3, patientName: 'Charlie', doctorId: 1; 4. id: 4, patientName: 'Diana', doctorId: 2.

```
[
  {
    "id": 1,
    "patientName": "Alice",
    "doctorId": 1
  },
  {
    "id": 2,
    "patientName": "Bob",
    "doctorId": 2
  },
  {
    "id": 3,
    "patientName": "Charlie",
    "doctorId": 1
  },
  {
    "id": 4,
    "patientName": "Diana",
    "doctorId": 2
  }
]
```

POST Method:

JSONRawHeaders7Test Results

Response Body

1 {
2 "message": "Appointment booked successfully",
3 "appointment": {
4 "id": 5,
5 "patientName": "Eve",
6 "doctorId": 1
7 }
8 }

POSThttp://localhost:3000/api/appointmentsSend

ParametersBodyHeadersAuthorizationPre-request ScriptPost-request Script

Content Typeapplication/jsonOverride

Raw Request Body

1 {
2 "patientName": "Eve",
3 "doctorId": 1
4 }
5

Status: 201 • 201 CreatedTime: 12 msSize: 342 B

JSONRawHeaders7Test Results

Response Body

1 {
2 "message": "Appointment booked successfully",
3 "appointment": {
4 "id": 5,
5 "patientName": "Eve",
6 "doctorId": 1
7 }
8 }

DELETE Method: /api/appointments/3

DELETE ⌵ <http://localhost:3000/api/appointments/3>

Parameters **Body** Headers Authorization Pre-req

Content Type [application/json](#) ⌵ [Override](#)

Raw Request Body

1	Raw Request Body
---	------------------

Status: **200** • **200 OK** Time: **114 ms** Size: **270 B**

JSON Raw Headers 7 Test Results

Response Body

1	{
2	
3	

```
{
  "message": "Appointment cancelled"
}
```

← ↻ ⓘ localhost:3000/api/appointments

📁 Import favorites |  Colorlib | Free Boo...

Pretty-print ☒

```
[
  {
    "id": 1,
    "patientName": "Alice",
    "doctorId": 1
  },
  {
    "id": 2,
    "patientName": "Bob",
    "doctorId": 2
  },
  {
    "id": 4,
    "patientName": "Diana",
    "doctorId": 2
  },
  {
    "id": 5,
    "patientName": "Eve",
    "doctorId": 1
  }
]
```

10. CRUD operations with MongoDB for Students Registration

Program code:

Backend:

config/db.js:

```
const mongoose = require("mongoose");
const connectDB = async () => {
  try {
    await
mongoose.connect("mongodb://127.0.0.1:27
017/authdb");
    console.log("MongoDB Connected");
  } catch (error) {
    console.error(error);
    process.exit(1);
  }
};
module.exports = connectDB;
```

models/User.js:

```
const mongoose = require("mongoose");
const userSchema = new
mongoose.Schema({
  name: String,
  email: String,
  password: String
});
module.exports = mongoose.model("User",
userSchema);
```

routes/auth.routes.js:

```
const express = require("express");
const router = express.Router();
const User = require("../models/User");
const jwt = require("jsonwebtoken");
// REGISTER
router.post("/register", async (req, res) => {
  const user = new User(req.body);
  await user.save();
  res.json({ message: "User registered
successfully" });
});
```

```
});
```

// LOGIN

```
router.post("/login", async (req, res) => {
  const user = await User.findOne({ email:
req.body.email });
  if (!user) {
    return res.status(401).json({ message:
"Invalid user" });
  }
}
```

```
  const token = jwt.sign({ id: user._id },
"secretkey", {
    expiresIn: "1h",
  });
```

```
  res.json({ token });
});
```

// VERIFY TOKEN API

```
router.get("/verify-token", (req, res) => {
  const authHeader =
req.headers.authorization;
  if (!authHeader) {
    return res.status(401).json({ message:
"Token missing" });
  }
  const token = authHeader.split(" ")[1];
  console.log("Auth Header:",
req.headers.authorization);
  console.log("Extracted Token:", token);
  try {
    const decoded = jwt.verify(token,
"secretkey");
    res.json({
      valid: true,
      userId: decoded.id,
    });
  } catch (err) {
    res.status(401).json({
      valid: false,
      message: "Invalid or expired token",
    });
  }
});
```

```

    });
  }
});
module.exports = router;
server.js:
const express = require("express");
const cors = require("cors");
const connectDB = require("./config/db");
const app = express();
app.use(cors());
app.use(express.json());
connectDB();
app.use("/api",
require("./routes/auth.routes"));
app.listen(5000, () => {
  console.log("Server running on port
5000");
});
Update package.json:
"scripts": {
  "test": "echo \ "Error" no test specified\ "
&& exit 1",
  "start" : "nodemon server.js",
  "dev": "nodemon server.js"
}

```

Frontend:

pages\Auth.css:

```

.auth-container {
  min-height: 100vh;
  display: flex;
  justify-content: center;
  align-items: center;
  background: linear-gradient(135deg,
#ff9f1c, #2ec4b6);
  font-family: "Segoe UI", Tahoma,
sans-serif;
}
/* Glass card */
.auth-card {
  background: rgba(255, 255, 255, 0.25);
  backdrop-filter: blur(12px);
  -webkit-backdrop-filter: blur(12px);
  border-radius: 16px;
  padding: 35px 40px;
  width: 100%;
  max-width: 380px;

```

```

  text-align: center;
  border: 1px solid rgba(255, 255, 255,
0.35);
  box-shadow: 0 8px 32px rgba(0, 0, 0,
0.25);
}
.auth-card h2 {
  margin-bottom: 20px;
  color: #1f2937;
  font-weight: 600;
  letter-spacing: 0.5px;
}
/* Inputs */
.auth-card input {
  width: 100%;
  padding: 12px 14px;
  margin-bottom: 16px;
  border-radius: 8px;
  border: none;
  outline: none;
  font-size: 14px;
  background: rgba(255, 255, 255, 0.7);
  color: #333;
}
.auth-card input::placeholder {
  color: #666;
}
.auth-card input:focus {
  box-shadow: 0 0 0 2px rgba(255, 159, 28,
0.6);
}
/* Button */
.auth-card button {
  width: 100%;
  padding: 12px;
  margin-top: 5px;
  background: linear-gradient(135deg,
#ff9f1c, #ffbf69);
  color: #fff;
  border: none;
  border-radius: 8px;
  font-size: 15px;
  font-weight: 600;
  cursor: pointer;
  transition: all 0.3s ease;
}
.auth-card button:hover {

```

```

transform: translateY(-2px);
background: linear-gradient(135deg,
#ff8c00, #ffa94d);
box-shadow: 0 6px 15px rgba(255, 159,
28, 0.5);
}
/* Links */
.switch-text {
margin-top: 18px;
font-size: 14px;
color: #1f2937;
}
.switch-text a,
.link {
color: #065f46;
cursor: pointer;
font-weight: 600;
text-decoration: none;
}
.switch-text a:hover,
.link:hover {
text-decoration: underline;
}
/* Error */
.error {
color: #b91c1c;
margin-bottom: 12px;
font-size: 14px;
font-weight: 500;
}

```

pages\Register.js:

```

import axios from "axios";
import { useState } from "react";
import { useNavigate, Link } from
"react-router-dom";
import "../Auth.css";
export default function Register() {
const [name, setName] = useState("");
const [email, setEmail] = useState("");
const [password, setPassword] =
useState("");
const [error, setError] = useState("");
const navigate = useNavigate();
const register = async () => {
try {

```

```

await
axios.post("http://localhost:5000/api/register
", {
name,
email,
password,
});
alert("Registered successfully");
navigate("/login");
} catch (err) {
setError("Registration failed. Try
again.");
}
};
return (
<div className="auth-container">
<div className="auth-card">
<h2>Create Account</h2>
{error && <p
className="error">{error}</p>}
<input
placeholder="Full Name"
value={name}
onChange={e =>
setName(e.target.value)}
/>
<input
type="email"
placeholder="Email"
value={email}
onChange={e =>
setEmail(e.target.value)}
/>
<input
type="password"
placeholder="Password"
value={password}
onChange={e =>
setPassword(e.target.value)}
/>
<button
onClick={register}>Register</button>
<p className="switch-text">
Already have an account?{" "}
<Link to="/login">Login</Link>
</p>
</div>

```



```

    </div>
  );
}
pages\Login.js
import axios from "axios";
import { useState } from "react";
import { useNavigate, Link } from
"react-router-dom";
import "../Auth.css";

export default function Login() {
  const [email, setEmail] = useState("");
  const [password, setPassword] =
useState("");
  const [error, setError] = useState("");
  const navigate = useNavigate();
  const login = async () => {
    try {
      const res = await
axios.post("http://localhost:5000/api/login",
{
    email,
    password,
  });

      localStorage.setItem("token",
res.data.token);
      navigate("/"); // Home page
    } catch (err) {
      setError("Invalid email or password");
    }
  };
  return (
    <div className="auth-container">
      <div className="auth-card">
        <h2>Login</h2>
        {error && <p
className="error">{error}</p>}
        <input
          type="email"
          placeholder="Email"
          value={email}
          onChange={e =>
setEmail(e.target.value)}
        />
        <input
          type="password"

```

```

          placeholder="Password"
          value={password}
          onChange={e =>
setPassword(e.target.value)}
        />
        <button
          onClick={login}>Login</button>
        <p className="switch-text">
          Don't have an account?{" "}
          <Link to="/register">Register</Link>
        </p>
      </div>
    </div>
  );
}

```

pages\Home.css:

```

.home-container {
  min-height: 100vh;
  display: flex;
  justify-content: center;
  align-items: center;
  background: linear-gradient(135deg,
#ff9f1c, #2ec4b6);
  font-family: "Segoe UI", Tahoma,
sans-serif;
}
/* Glass card */
.home-card {
  background: rgba(255, 255, 255, 0.25);
  backdrop-filter: blur(12px);
  -webkit-backdrop-filter: blur(12px);
  padding: 45px 40px;
  width: 100%;
  max-width: 440px;
  border-radius: 18px;
  text-align: center;
  border: 1px solid rgba(255, 255, 255,
0.35);
  box-shadow: 0 8px 32px rgba(0, 0, 0,
0.25);
}
.home-card h2 {
  margin-bottom: 12px;
  color: #1f2937;
  font-size: 26px;
  font-weight: 600;

```

```

}
.home-card p {
  color: #1f2937;
  margin-bottom: 30px;
  font-size: 15px;
  opacity: 0.9;
}
/* Logout button */
.home-card button {
  padding: 12px 32px;
  background: linear-gradient(135deg,
#ff9f1c, #ffbf69);
  color: #fff;
  border: none;
  border-radius: 10px;
  cursor: pointer;
  font-size: 15px;
  font-weight: 600;
  transition: all 0.3s ease;
}
.home-card button:hover {
  transform: translateY(-2px);
  background: linear-gradient(135deg,
#ff8c00, #ffa94d);
  box-shadow: 0 6px 15px rgba(255, 159,
28, 0.5);
}

```

pages\Home.js:

```

import axios from "axios";
import { useEffect } from "react";
import { useNavigate } from
"react-router-dom";
import "./Home.css";
export default function Home() {
  const navigate = useNavigate();
  useEffect(() => {
    const verify = async () => {
      try {
        const token=
localStorage.getItem("token")
        await
axios.get("http://localhost:5000/api/verify-to
ken", {
          headers: {
            Authorization: `Bearer ${token}`,
          },
        },

```

```

      });
    } catch {
      localStorage.removeItem("token");
      navigate("/login");
    }
  };
  verify();
}, [navigate]);
const logout = () => {
  localStorage.removeItem("token");
  navigate("/login");
};
return (
  <div className="home-container">
    <div className="home-card">
      <h2>Welcome 🙌</h2>
      <p>You are successfully logged
in.</p>
      <button
onClick={logout}>Logout</button>
    </div>
  </div>
);
}

```

App.js:

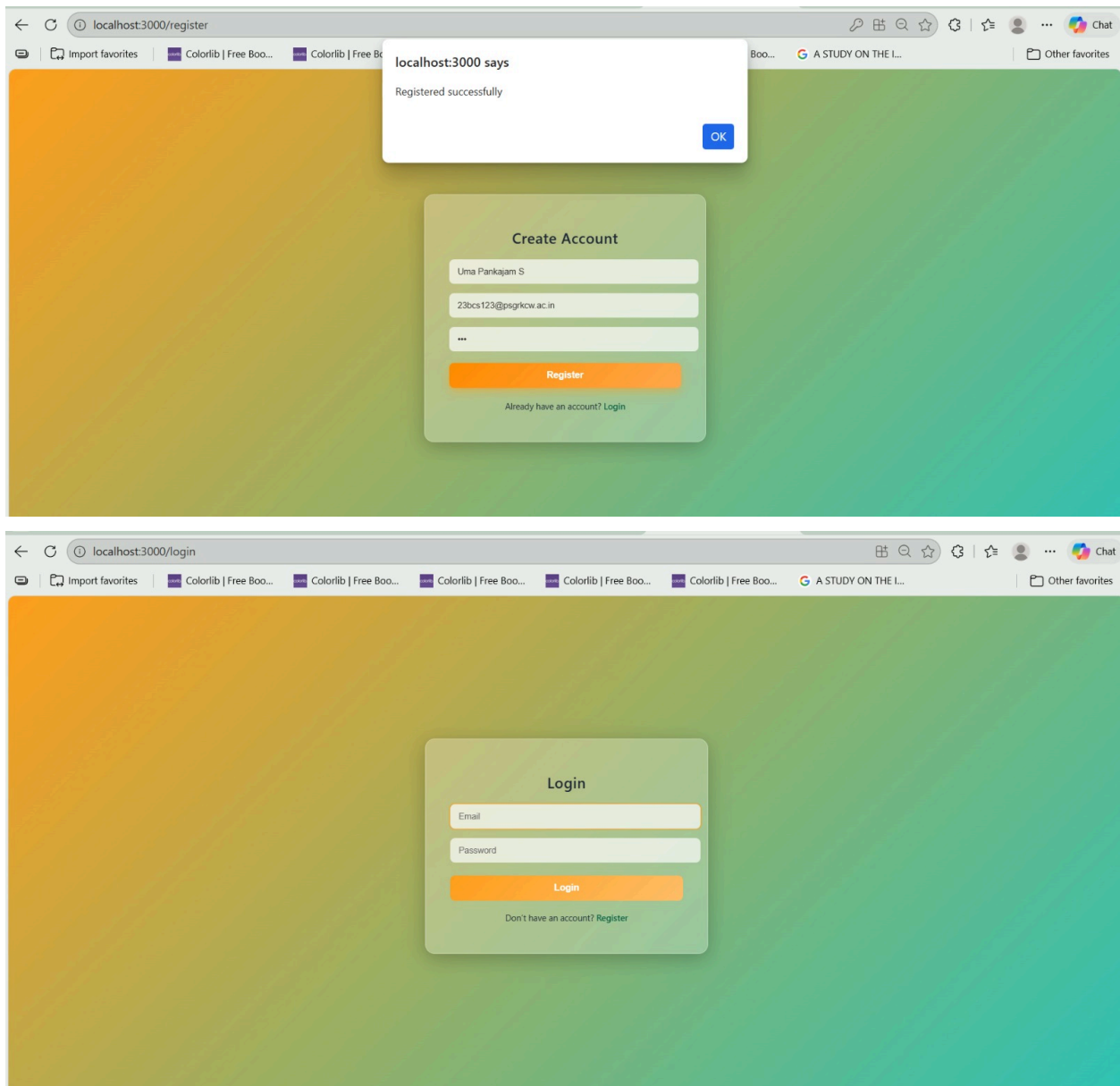
```

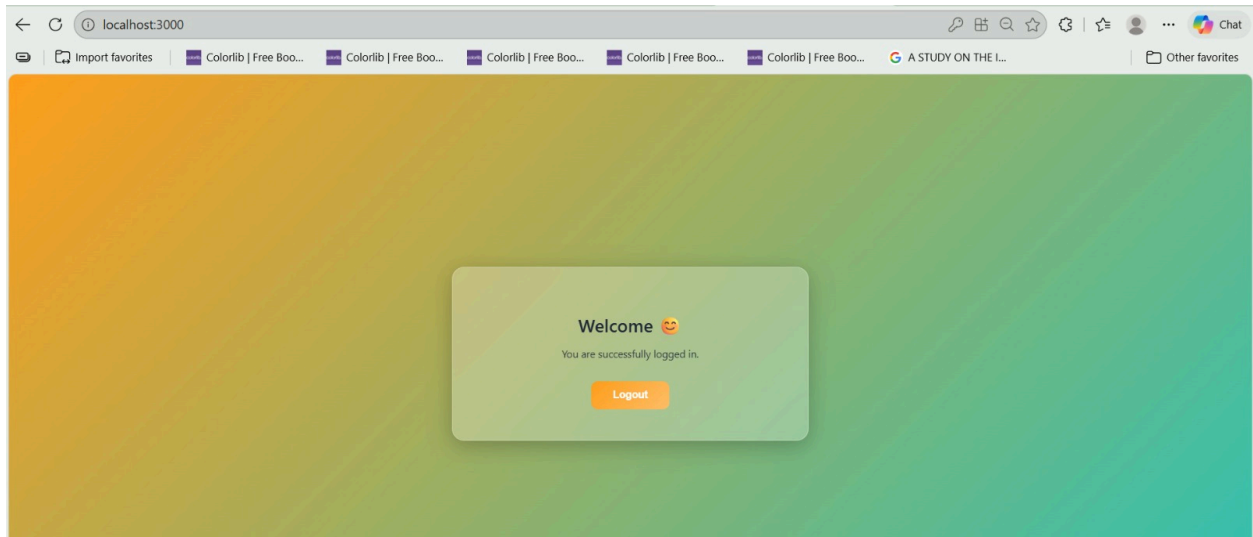
import { BrowserRouter, Routes, Route,
Navigate } from "react-router-dom";
import Login from "../pages/Login";
import Register from "../pages/Register";
import Home from "../pages/Home";
function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Home />}
/>
        <Route path="/login"
element={<Login />} />
        <Route path="/register"
element={<Register />} />
      </Routes>
    </BrowserRouter>
  );
}

export default App;

```

Output:





Cluster0 > test > users

Documents 1 Aggregations Schema Indexes 1 Validation

Type a query: { field: 'value' } or [Generate query](#) ✨

Explain Reset Find Options ▾

ADD DATA UPDATE DELETE EXPORT CODE

25 1 - 1 of 1

```
_id: ObjectId('699597c919197dba15edcfba')
name: "Uma Pankajam S"
email: "23bcs123@psgrkcw.ac.in"
password: "123"
__v: 0
```

11. Online Examination Portal: Authentication and Session Management

Using JWT in Node.js Applications

Program Coding:

index.html:

```
<!DOCTYPE html>
<html>
<head>
  <title>Online Exam Portal</title>
  <style>
    body {
      font-family: Arial;
      background-color: #f2f2f2;
      text-align: center;
      margin-top: 40px;
    }
    .container {
      background: white;
      width: 400px;
      margin: auto;
      padding: 20px;
      border-radius: 10px;
      box-shadow: 0px 0px 10px gray;
    }
    input {
      width: 90%;
      padding: 8px;
      margin: 8px 0;
    }
    button {
      padding: 8px 15px;
      margin-top: 10px;
      cursor: pointer;
    }
    hr {
      margin: 20px 0;
    }
    #examSection {
      display: none;
      text-align: left;
    }
  </style>
</head>
<body>
<div class="container">
```

```
<h2>Online Exam Portal</h2>
<!-- Register -->
<h3>Register</h3>
<input type="text" id="regUsername"
placeholder="Username"><br>
<input type="password" id="regPassword"
placeholder="Password"><br>
<button
onclick="register()">Register</button>
<hr>
<!-- Login -->
<h3>Login</h3>
<input type="text" id="loginUsername"
placeholder="Username"><br>
<input type="password"
id="loginPassword"
placeholder="Password"><br>
<button onclick="login()">Login</button>
<p id="message"></p>
<hr>
<!-- Exam Section -->
<div id="examSection">
  <h3>Exam Questions</h3>
  <div id="questions"></div>
  <button onclick="submitExam()">Submit
Exam</button>
  <button
onclick="logout()">Logout</button>
</div>
</div>
<script>
const API_URL = "http://localhost:5000";
let correctAnswers = {};
// REGISTER
async function register() {
  const username =
document.getElementById("regUsername").
value;
  const password =
document.getElementById("regPassword").
value;
```

```

    const res = await fetch(API_URL +
"/register", {
    method: "POST",
    headers: { "Content-Type":
"application/json" },
    body: JSON.stringify({ username,
password })
});
    const data = await res.json();

document.getElementById("message").innerHTML
Text = data.message;
}
async function login() {
    const username =
document.getElementById("loginUsername
").value;
    const password =
document.getElementById("loginPassword"
).value;
    const res = await fetch(API_URL +
"/login", {
    method: "POST",
    headers: { "Content-Type":
"application/json" },
    body: JSON.stringify({ username,
password })
});
    const data = await res.json();
    if (data.token) {
        localStorage.setItem("token",
data.token);

document.getElementById("message").innerHTML
Text = "Login successful!";
        loadExam();
    } else {

document.getElementById("message").innerHTML
Text = data.message;
    }
}
async function loadExam() {
    const token =
localStorage.getItem("token");
    const res = await fetch(API_URL +
"/exam", {

```

```

    method: "GET",
    headers: {
        "Authorization": "Bearer " + token
    }
});
const data = await res.json();
if (data.exam) {

document.getElementById("examSection").
style.display = "block";
    let output = "";
    correctAnswers = {};
    data.exam.forEach(q => {
        correctAnswers[q.id] = q.answer;
        output += `<div>
            <p><b>Q${q.id}:
${q.question}</b></p>`;
        q.options.forEach(option => {
            output += `
            <label>
            <input type="radio"
name="q${q.id}" value="${option}">
            ${option}
            </label><br>
            `;
        });
        output += `</div><br>`;
    });

document.getElementById("questions").inner
HTML = output;
    } else {

document.getElementById("message").innerHTML
Text = "Unauthorized!";
    }
}
function submitExam() {
    let score = 0;
    let total =
Object.keys(correctAnswers).length;

    for (let q in correctAnswers) {
        const selected =
document.querySelector(`input[name="q${q
}"]:checked`);

```

```

        if (selected && selected.value ===
correctAnswers[q]) {
            score++;
        }
    }
    alert("Your Score: " + score + " / " +
total);
function logout() {
    localStorage.removeItem("token");
    location.reload();
}
</script>
</body>
</html>

```

server.js:

```

const express = require("express");
const jwt = require("jsonwebtoken");
const bcrypt = require("bcryptjs");
const cors = require("cors");
const path = require("path");
const app = express();
const PORT = 5000;
const SECRET_KEY = "exam_secret_key";
// Middleware
app.use(express.json());
app.use(cors());
app.use(express.static(__dirname)); // Serves
index.html
// Temporary in-memory storage
let users = [];
app.get("/", (req, res) => {
    res.send("Online Exam Portal API is
Running 🚀");
});
app.post("/register", async (req, res) => {
    const { username, password } = req.body;
    if (!username || !password) {
        return res.status(400).json({ message:
"All fields required" });
    }
    const userExists = users.find(u =>
u.username === username);
    if (userExists) {
        return res.status(400).json({ message:
"User already exists" });
    }

```

```

        const hashedPassword = await
bcrypt.hash(password, 10);
        users.push({ username, password:
hashedPassword });
        res.json({ message: "User registered
successfully" });
    });
    app.post("/login", async (req, res) => {
        const { username, password } = req.body;

        const user = users.find(u => u.username
=== username);
        if (!user) {
            return res.status(400).json({ message:
"Invalid username" });
        }
        const validPassword = await
bcrypt.compare(password, user.password);
        if (!validPassword) {
            return res.status(400).json({ message:
"Invalid password" });
        }
        const token = jwt.sign(
            { username: user.username },
            SECRET_KEY,
            { expiresIn: "1h" }
        );
        res.json({ message: "Login successful",
token });
    });
    function verifyToken(req, res, next) {
        const header =
req.headers["authorization"];
        const token = header && header.split("
")[1];
        if (!token) {
            return res.status(401).json({ message:
"Access denied. No token provided." });
        }
        jwt.verify(token, SECRET_KEY, (err,
decoded) => {
            if (err) {
                return res.status(403).json({ message:
"Invalid token" });
            }
            req.user = decoded;
            next();
        }
    }

```

```

    });
  }
  app.get("/exam", verifyToken, (req, res) => {
    res.json({
      message: `Welcome
${req.user.username}`,
      exam: [
        {
          id: 1,
          question: "What is 5 + 5?",
          options: ["8", "9", "10", "11"],
          answer: "10"
        },
        {
          id: 2,

```

```

          question: "What does JWT stand for?",
          options: [
            "Java Web Token",
            "JSON Web Token",
            "JavaScript Web Token",
            "Joint Web Token"
          ],
          answer: "JSON Web Token"
        }
      ]
    });
  });
  app.listen(PORT, () => {
    console.log(`Server running at
http://localhost:${PORT}`);
  });

```


Output:

Online Exam Portal

Register

Register

Login

Login

User registered successfully

localhost:5000 says

Your Score: 2 / 2

OK

Exam Questions

Q1: What is 5 + 5?

☐

8

☐

9

☒

10

☐

11

Q2: What does JWT stand for?

☐

Java

☒

Web Token

☐

JSON Web Token

☐

JavaScript Web Token

☐

Web Token

Joint

Submit Exam

Logout

12. Placement Registration Website Using React.js Frontend with Node.js Backend APIs

Program Coding:

server.js:

```
const express = require("express");
const cors = require("cors");
const fs = require("fs");
const path = require("path");
const app = express();
app.use(cors());
app.use(express.json());
const filePath = path.join(__dirname,
"students.json");

// Make sure file exists
if (!fs.existsSync(filePath)) {
  fs.writeFileSync(filePath, "[]");
}

app.post("/register", (req, res) => {
  try {
    const newStudent = req.body;

    const data = fs.readFileSync(filePath);
    const students = JSON.parse(data);

    students.push(newStudent);

    fs.writeFileSync(filePath,
JSON.stringify(students, null, 2));

    res.status(200).send("Student Registered
Successfully");
  } catch (error) {
    console.log("ERROR:", error);
    res.status(500).send("Server Error");
  }
});
```

```
app.listen(5000, () => {
  console.log("Server running on port
5000");
});
```

app.js:

```
import React, { useState } from "react";
import axios from "axios";

function App() {
  const [formData, setFormData] =
useState({
  name: "",
  email: "",
  department: ""
});

  const handleChange = (e) => {
    setFormData({ ...formData,
[e.target.name]: e.target.value });
  };

  const handleSubmit = async (e) => {
    e.preventDefault();
    await
axios.post("http://localhost:5000/register",
formData);
    alert("Registered Successfully");
  };

  return (
    <div style={{ textAlign: "center" }}>
      <h2>Placement Registration</h2>
      <form onSubmit={handleSubmit}>
```

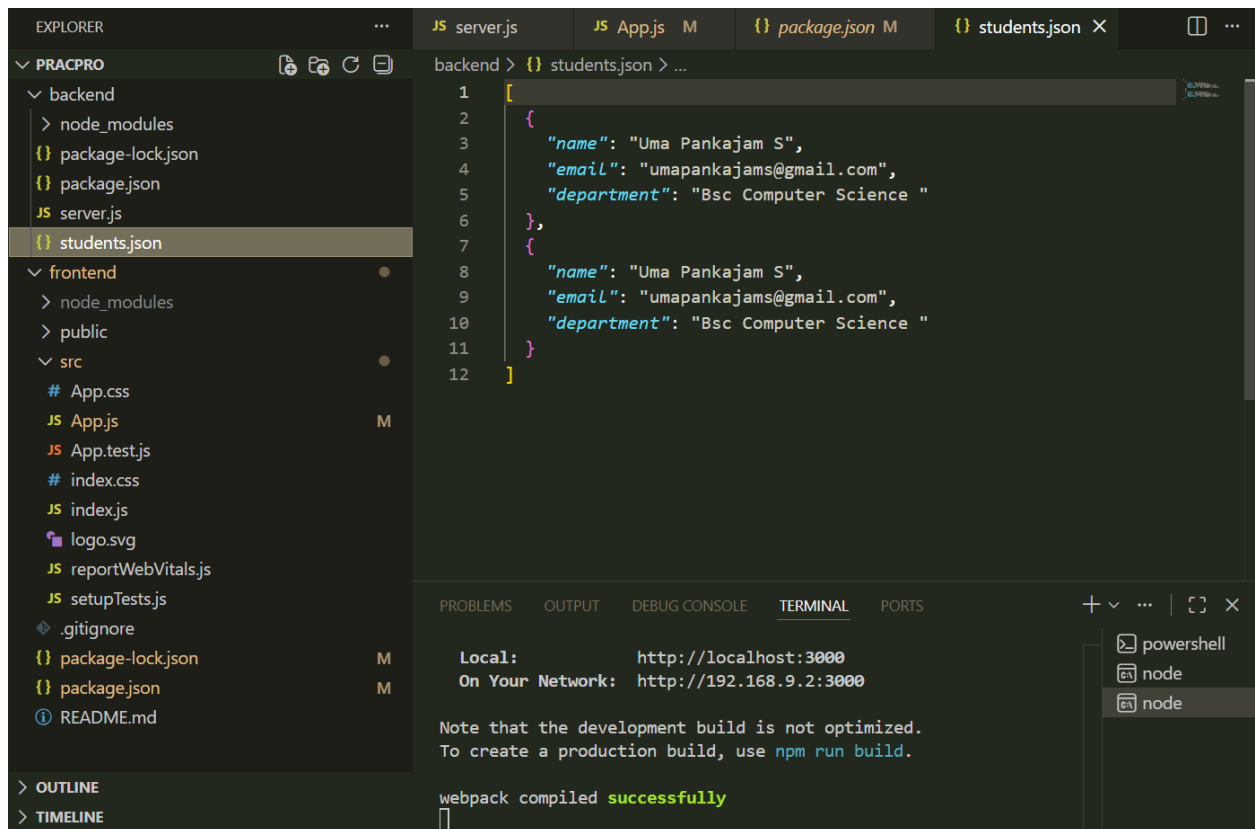
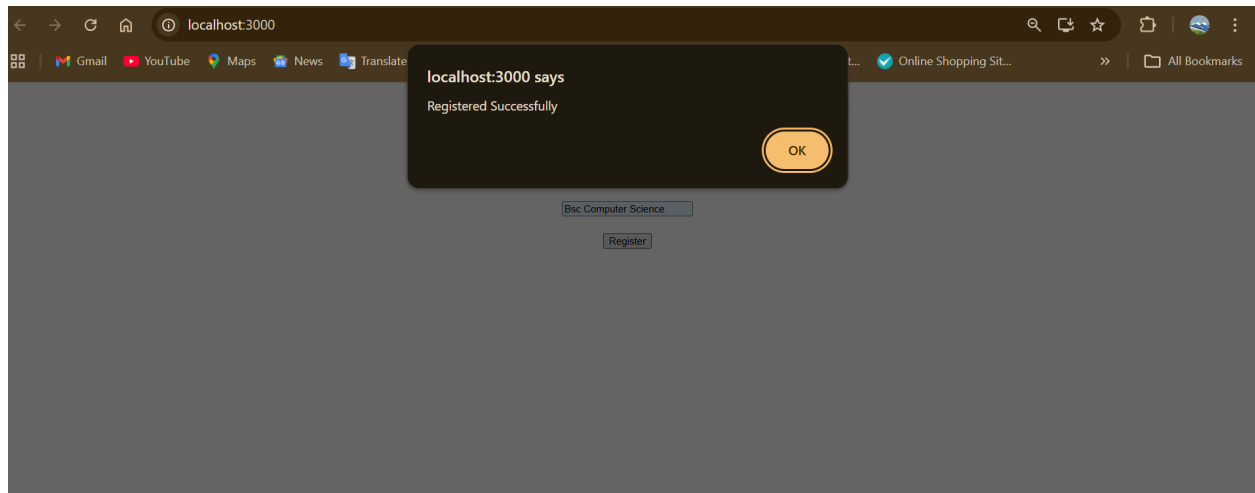
```

        <input name="name"
placeholder="Name"
onChange={handleChange} required /><br
/><br />
        <input name="email"
placeholder="Email"
onChange={handleChange} required /><br
/><br />
        <input name="department"
placeholder="Department"
onChange={handleChange} required /><br
/><br />
        <button
type="submit">Register</button>
    </form>
</div>
);
}

export default App;

```

Output:



13. Deployment of a Full-Stack Student Admission Application on a Cloud Platform

Program Coding:

app.jsx

```
import { useState, useEffect } from
"react";
import axios from "axios";
export default function App() {
  const [name, setName] =
useState("");
  const [phone, setPhone] =
useState("");
  const [students, setStudents] =
useState([]);
  const API =
"https://student-admission-backend.o
nrender.com";
  const fetchStudents = async () => {
    const res = await
axios.get(`${API}/users`);
    setStudents(res.data);
  };
  const addStudent = async () => {
    if (!name || !phone) return;
    await
axios.post(`${API}/add-user`, {
name, phone });
    setName("");
    setPhone("");
    fetchStudents();
  };
  const deleteStudent = async (id) =>
{
    await
axios.delete(`${API}/delete-user/${i
d}`);
    fetchStudents();
  };
  useEffect(() => {
    fetchStudents();
  }, []);
  return (
    <div style={{ padding: 40 }}>
      <h2>&img alt="graduation cap icon" data-bbox="285 865 305 885"/> Student Admission
List</h2>
```

```
<input
  placeholder="Student Name"
  value={name}
  onChange={(e) =>
setName(e.target.value)}
/>
<br /><br />
<input
  placeholder="Phone Number"
  value={phone}
  onChange={(e) =>
setPhone(e.target.value)}
/>
<br /><br />
      <button
onClick={addStudent}>Add
Student</button>
      <h3>Admission Records</h3>
      <ul>
        {students.map((s) => (
          <li key={s.id}>
            {s.name} — {s.phone}
            <button
              style={{ marginLeft: 10 }}
              onClick={() =>
deleteStudent(s.id)} >
              Remove
            </button>
          </li>
        ))}
      </ul>
    </div>
  ); }
```

main.jsx

```
import { StrictMode } from 'react'
import { createRoot } from
'react-dom/client'
import './index.css'
import App from './App.jsx'
```

```

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)

```

server.js

```

const express = require("express");
const cors = require("cors");
const sqlite3 = require("sqlite3").verbose();
const app = express();
// Middleware
app.use(cors());
app.use(express.json());
// Connect SQLite Database
const db = new sqlite3.Database("./database.db",
(err) => {
  if (err) {
    console.log(err.message);
  } else {
    console.log("Connected to SQLite database");
  }
});
// Create Students Table
db.run(
  `CREATE TABLE IF NOT EXISTS
  students (
    id INTEGER PRIMARY KEY
    AUTOINCREMENT,
    name TEXT,
    phone TEXT
  )`
);
// + Add Student
app.post("/add-user", (req, res) => {
  const { name, phone } = req.body;
  if (!name || !phone) {
    return res.status(400).json({ error:
    "Name and phone required" });
  }
  db.run(

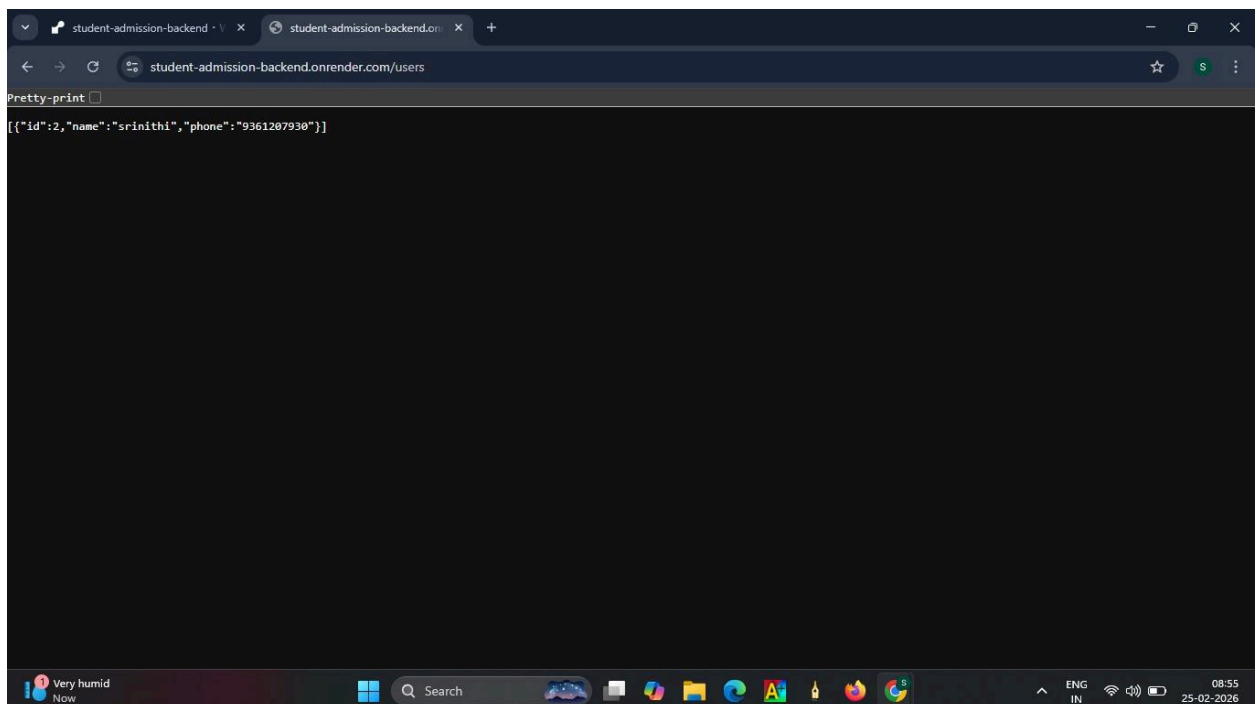
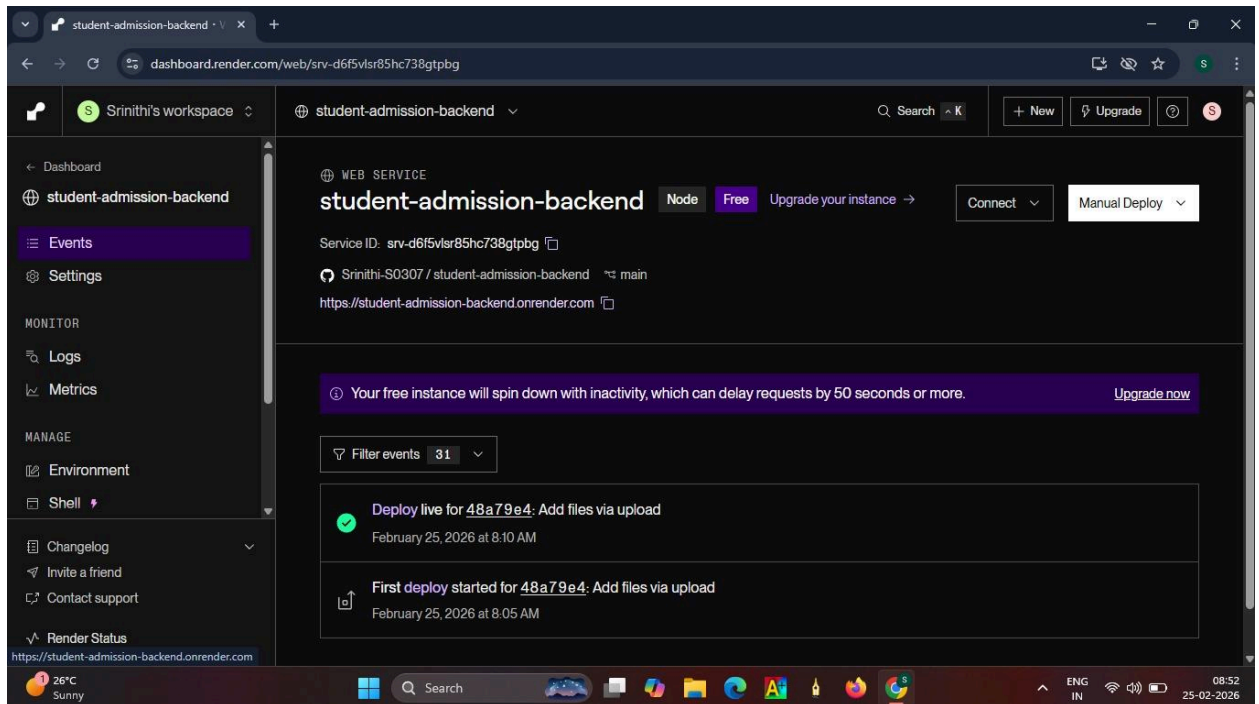
```

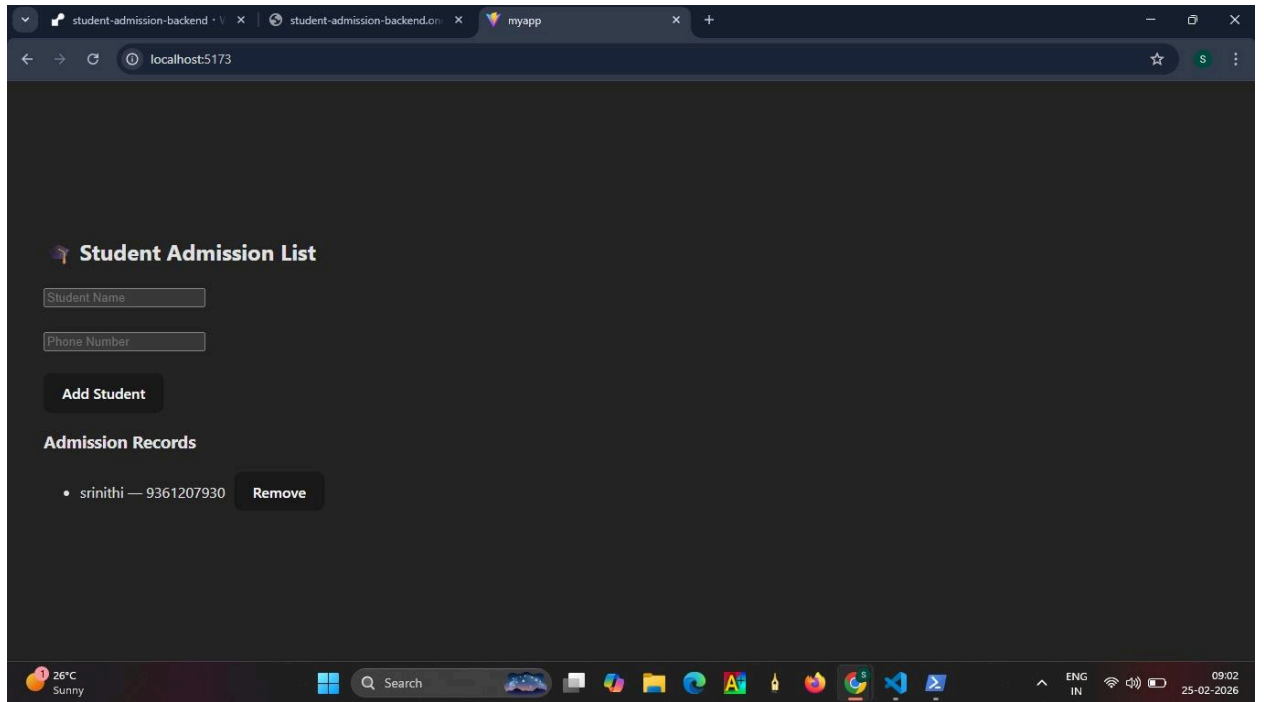
```

    "INSERT INTO students(name,
    phone) VALUES(?, ?)",
    [name, phone],
    function (err) {
      if (err) {
        return res.status(500).json(err);
      }
      res.json({
        message: "Student added
        successfully",
        id: this.lastID,
      });
    });
// 📋 Get All Students
app.get("/users", (req, res) => {
  db.all("SELECT * FROM
  students", [], (err, rows) => {
    if (err) {
      return res.status(500).json(err);
    }
    res.json(rows);
  });
});
// ✗ Delete Student
app.delete("/delete-user/:id", (req,
res) => {
  const id = req.params.id;
  db.run("DELETE FROM students
  WHERE id = ?", [id], function (err)
  {
    if (err) {
      return res.status(500).json(err);
    }
    res.json({ message: "Student
    removed successfully" });
  });
});
// Start Server
app.listen(5000, () => {
  console.log("Backend running on
  http://localhost:5000");
});

```

Output:





14. Performance Profiling and Debugging of a Single Page Application Using React.js

Program Coding:

main.jsx:

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";
import "./App.css";
```

```
ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

App.jsx:

```
import { useState } from "react";
import Counter from "./Counter";
import ExpensiveComponent from
"./ExpensiveComponent";
import "./App.css";
```

```
function App() {
  const [count, setCount] = useState(0);
  const [text, setText] = useState("");

  console.log("App Rendered");

  return (
    <div className="container">
      <h1>⚡ Performance Debugging Demo</h1>

      <Counter count={count}
setCount={setCount} />

      <input
type="text"
```

```
placeholder="Type something..."
value={text}
onChange={(e) =>
setText(e.target.value)}
/>
```

```
<ExpensiveComponent />
</div>
);
}
```

```
export default App;
```

Counter.jsx

```
function Counter({ count, setCount }) {
  console.log("Counter Rendered");

  return (
    <div className="box">
      <h2>Count: {count}</h2>
      <button onClick={() => setCount(count
+ 1)}>
        Increase
      </button>
    </div>
  );
}
```

```
export default Counter;
```

ExpensiveComponent.jsx (Before Optimization)

```
function ExpensiveComponent() {
  console.log("Expensive Component
Rendered");

  let total = 0;
  for (let i = 0; i < 1000000000; i++) {
```

```

    total += i;
  }

  return (
    <div className="box">
      <h3>Heavy Calculation Done</h3>
    </div>
  );
}

export default ExpensiveComponent;
App.css:
body {
  background-color: #f4f6f8;
}

.container {
  width: 600px;
  margin: 40px auto;
  padding: 20px;
  border: 2px solid #333;
  background-color: white;
}

.box {
  margin: 15px 0;
  padding: 10px;
  border: 1px solid #ccc;
}

```

```

//Optimization Using React.memo
ExpensiveComponent.jsx (After Optimization)
import React from "react";

function ExpensiveComponent() {
  console.log("Expensive Component Rendered");

  let total = 0;
  for (let i = 0; i < 1000000000; i++) {
    total += i;
  }

  return (
    <div className="box">
      <h3>Heavy Calculation Done</h3>
    </div>
  );
}

export default
React.memo(ExpensiveComponent);

```

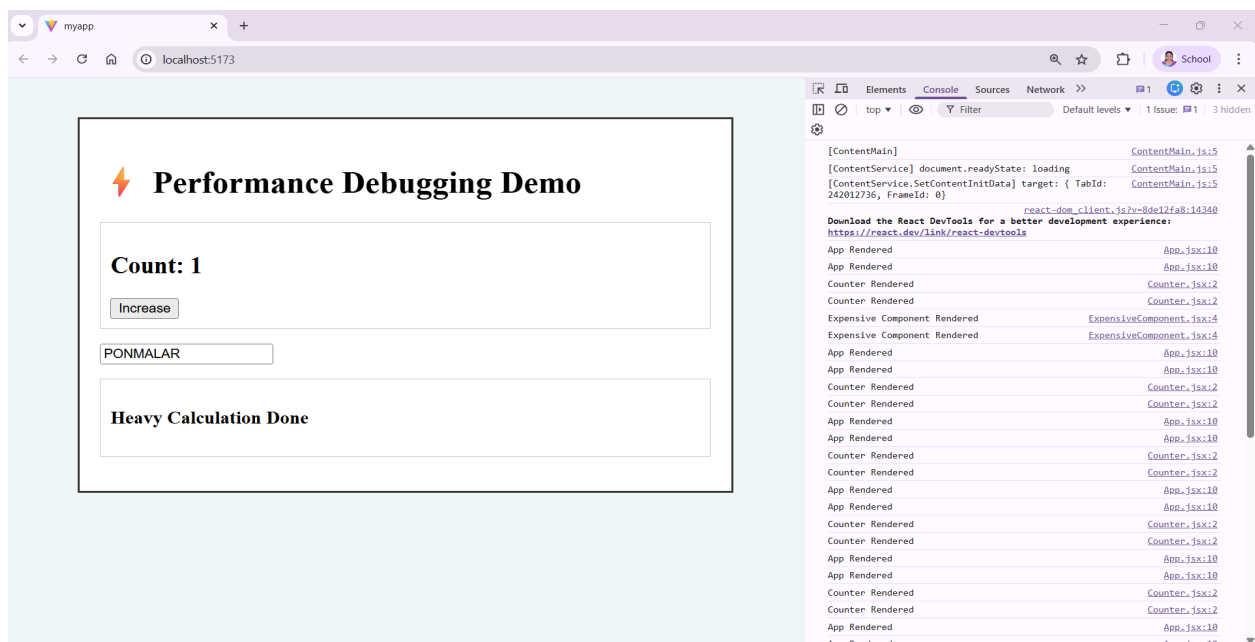
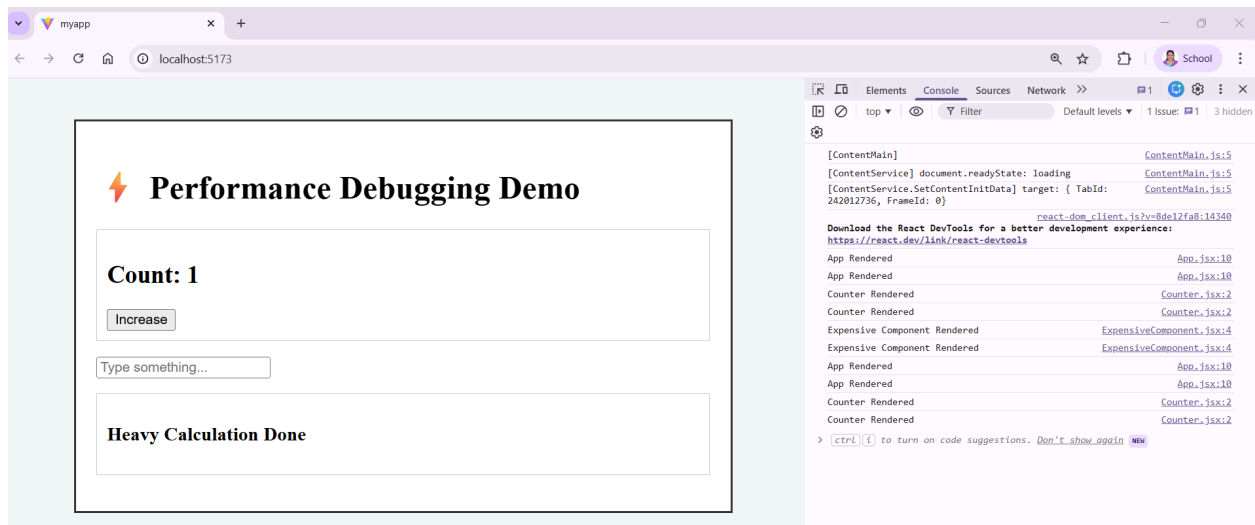
Output: (Before Optimization)

The image displays two screenshots of a web application running on localhost:5173, illustrating performance debugging results before optimization. The application features a title "Performance Debugging Demo" with a lightning bolt icon. It includes a "Count: 0" display, an "Increase" button, a text input field, and a "Heavy Calculation Done" message.

In the first screenshot, the text input field is empty. In the second screenshot, the text input field contains the text "PONMALAR".

The Chrome DevTools Console on the right shows a list of log messages, including "App Rendered", "Counter Rendered", and "Expensive Component Rendered", indicating the sequence of component updates.

(After Optimization)



Performance Comparison

Feature	Before	After
Expensive Component Render	Every time	Only once
UI Speed	Slow	Smooth
Heavy Loop Execution	Multiple times	Single time

15. React To-Do App with Git Version Control

Git deployment

PART 1 — Initial Setup (Project Owner)

Step 1: Install Requirements

Make sure you have:

- Node.js
- Git
- VS Code
- GitHub account

Check versions:

`node -v`

`git --version`

Step 2: Create React App

Open VS Code → Open Terminal:

`npx create-react-app react-todo-app`

`cd react-todo-app`

`code .`

Start app:

`npm start`

Step 3: Initialize Git

Inside project folder:

`git init`

Check status:

`git status`

Step 4: First Commit

`git add .`

`git commit -m "Initial React app setup"`

Step 5: Create GitHub Repository

Go to GitHub → New Repository →

Name it:

`react-todo-app`

Then connect local project:

`git remote add origin`

`https://github.com/yourusername/react-todo-app.git`

`git branch -M main`

`git push -u origin main`

Program Coding:

App.jsx:

```
import { useState } from "react";

function App() {
  // State for current input
  const [task, setTask] = useState("");
  // State for list of tasks
  const [tasks, setTasks] = useState([]);

  // Add task
  const addTask = () => {
    if (task.trim() === "") return;
    setTasks([...tasks, task]);
    setTask("");
  };

  // Delete task
  const deleteTask = (indexToDelete) => {
    setTasks(tasks.filter((_, index) => index !== indexToDelete));
  };

  return (
    <div style={{ textAlign: "center",
marginTop: "50px" }}>
      <h1>React To-Do App</h1>

      <div style={{ marginBottom: "20px"
}}>
        <input
          type="text"
          placeholder="Enter a task"
          value={task}
          onChange={(e) =>
            setTask(e.target.value)}
          style={{ padding: "8px", width:
"200px" }}
        />
        <button
          onClick={addTask}
          style={{
            padding: "8px 16px",
            marginLeft: "10px",
```

```

        cursor: "pointer",
      }}
    >
      Add
    </button>
  </div>

  <ul style={{ listStyleType: "none",
padding: 0 }}>
    {tasks.map((t, index) => (
      <li
        key={index}
        style={{
          marginBottom: "10px",
          display: "flex",
          justifyContent: "center",
          alignItems: "center",
          gap: "10px",
        }}
      >
        <span>{t}</span>
        <button
          onClick={() => deleteTask(index)}
          style={{
            padding: "4px 8px",
            cursor: "pointer",
            backgroundColor: "#ff4d4f",
            color: "white",

```

```

        border: "none",
        borderRadius: "4px",
      }}
    >
      Delete
    </button>
  </li>
  )))
</ul>
</div>
);
}

```

export default A

Main.jsx:

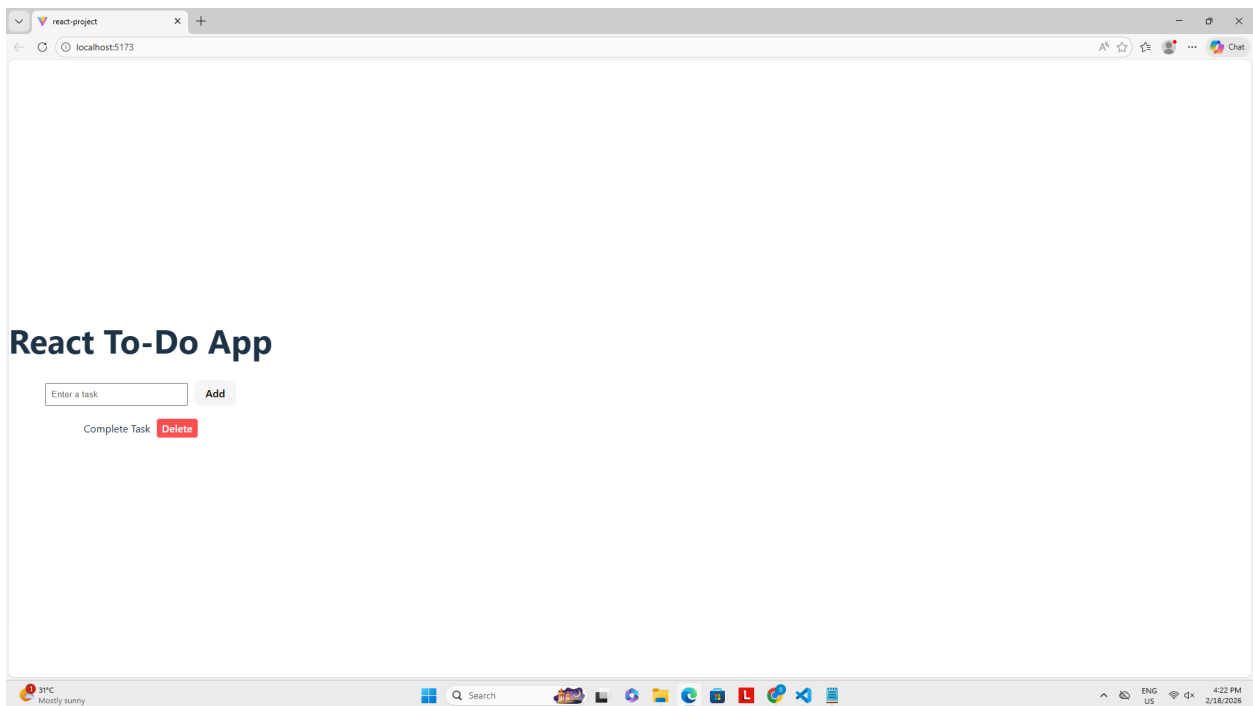
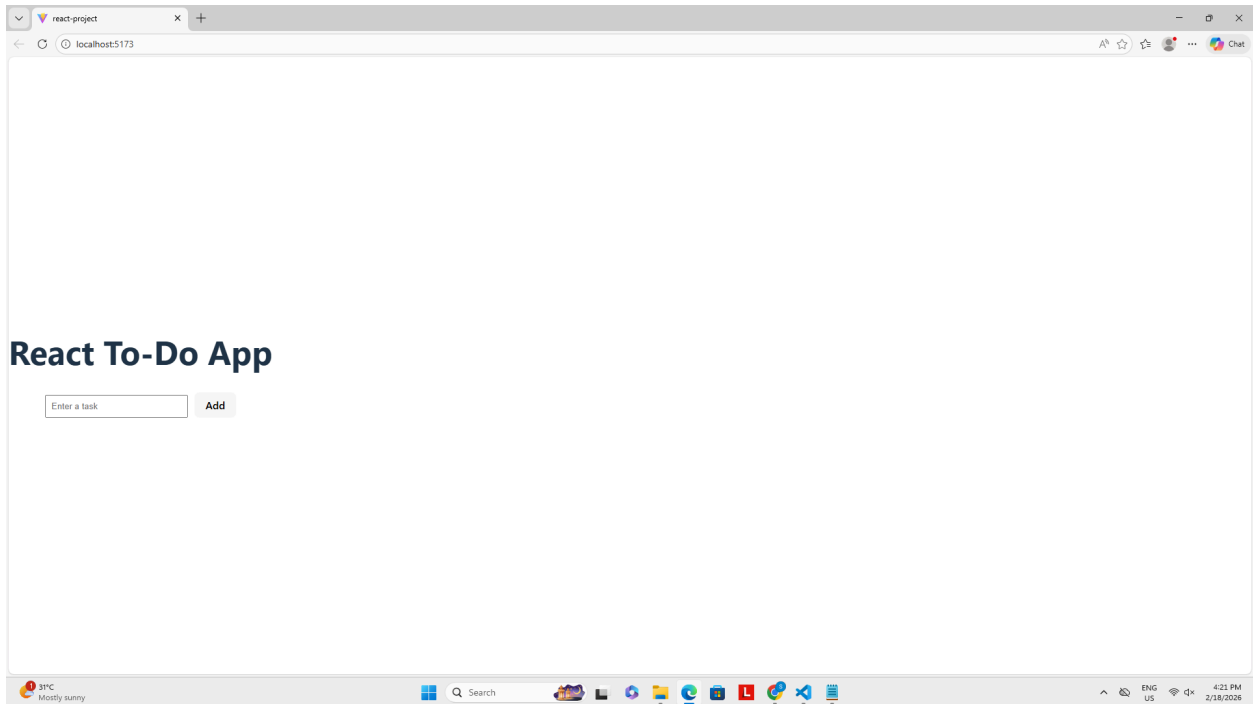
```

import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";
import "./index.css"; // optional styling

ReactDOM.createRoot(document.getElemen
tById("root")).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);

```

Output:



react-todo-app

unknown and unknown Initialize project using Create React App 3df1e73 · 1 hour ago 1 Commit

public	Initialize project using Create React App	1 hour ago
src	Initialize project using Create React App	1 hour ago
.gitignore	Initialize project using Create React App	1 hour ago
README.md	Initialize project using Create React App	1 hour ago
package-lock.json	Initialize project using Create React App	1 hour ago
package.json	Initialize project using Create React App	1 hour ago

README

Getting Started with Create React App

This project was bootstrapped with [Create React App](#).

Available Scripts

In the project directory, you can run:

npm start

Runs the app in the development mode.
Open <http://localhost:3000> to view it in your browser.

About

No description, website, or topics provided.

Readme
Activity
0 stars
0 watching
0 forks

Releases

No releases published
[Create a new release](#)

Packages

No packages published
[Publish your first package](#)

Languages

JavaScript 41.9%
HTML 37.7%
CSS 20.4%

Suggested workflows

Based on your tech stack

Deno
Test your Deno project

react-todo-app

Getting Started with Create React App

This project was bootstrapped with [Create React App](#).

Available Scripts

In the project directory, you can run:

npm start

Runs the app in the development mode.
Open <http://localhost:3000> to view it in your browser.

The page will reload when you make changes.
You may also see any lint errors in the console.

npm test

Launches the test runner in the interactive watch mode.
See the section about [running tests](#) for more information.

npm run build

Builds the app for production to the `build` folder.
It correctly bundles React in production mode and optimizes the build for the best performance.

The build is minified and the filenames include the hashes.
Your app is ready to be deployed!

See the section about [deployment](#) for more information.

npm run eject

Note: this is a one-way operation. Once you `eject`, you can't go back!

If you aren't satisfied with the build tool and configuration choices, you can `eject` at any time. This command will remove the single build dependency from your project.

Instead, it will copy all the configuration files and the transitive dependencies (webpack, Babel, ESLint, etc) right into your project so you have full control over them. All of the commands except `test` will still work, but they will take some time to get up to speed.

no packages published
[Publish your first package](#)

Languages

JavaScript 41.9%
HTML 37.7%
CSS 20.4%

Suggested workflows

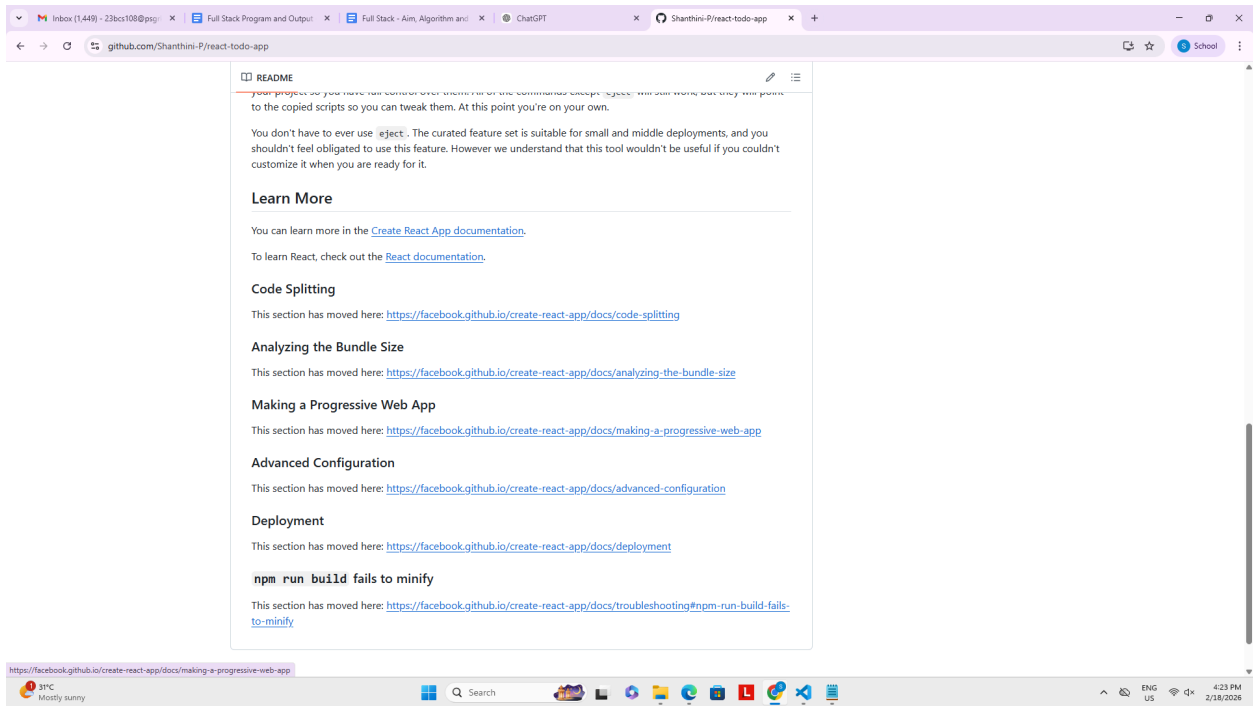
Based on your tech stack

Deno
Test your Deno project

SLSA Generic generator
Generate SLSA3 provenance for your existing release workflows

Publish Node.js Package to GitHub Packages
Publishes a Node.js package to GitHub Packages

More workflows
Dismiss suggestions



16. Build and Deploy a Full Stack Web Application – Online Shopping

Program Coding:

Backend:

```
mkdir backend
```

```
cd backend
```

```
npm init -y
```

```
npm install express mongoose cors
```

```
npm install nodemon --save-dev
```

server.js:

```
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");
```

```
const app = express();
app.use(cors());
app.use(express.json());
```

```
mongoose.connect("YOUR_MONGODB_ATLAS_URL")
.then(() => console.log("MongoDB Connected"))
.catch(err => console.log(err));
```

```
// Product Schema
const Product = mongoose.model("Product",
{
  name: String,
  price: Number,
  image: String
});
```

```
// Get products
app.get("/products", async (req, res) => {
  const products = await Product.find();
  res.json(products);
```

```
});
```

```
// Add product (for testing)
app.post("/products", async (req, res) => {
  const newProduct = new
Product(req.body);
  await newProduct.save();
  res.json(newProduct);
});
```

```
app.listen(5000, () => console.log("Server
running on port 5000"));
```

Run backend:

```
npx nodemon server.js
```

Frontend:

```
npx create-react-app frontend
cd frontend
npm install axios
```

App.js:

```
import React, { useState, useEffect } from
"react";
import "./App.css";
```

```
const productsData = [
  { id: 1, name: "Shoes", price: 1500, image:
"https://via.placeholder.com/150" },
  { id: 2, name: "T-Shirt", price: 800, image:
"https://via.placeholder.com/150" },
  { id: 3, name: "Watch", price: 2500, image:
"https://via.placeholder.com/150" },
];
```

```
function App() {
```

```

const [cart, setCart] = useState([]);
const [search, setSearch] = useState("");
const [showCheckout, setShowCheckout]
= useState(false);

// Load cart from localStorage
useEffect(() => {
  const savedCart =
JSON.parse(localStorage.getItem("cart"));
  if (savedCart) setCart(savedCart);
}, []);

// Save cart to localStorage
useEffect(() => {
  localStorage.setItem("cart",
JSON.stringify(cart));
}, [cart]);

const addToCart = (product) => {
  const existing = cart.find((item) =>
item.id === product.id);
  if (existing) {
    setCart(cart.map((item) =>
      item.id === product.id
        ? { ...item, quantity: item.quantity + 1
        : item
    ));
  } else {
    setCart([...cart, { ...product, quantity: 1
  }]);
  }
};

const increaseQty = (id) => {
  setCart(cart.map(item =>
    item.id === id ? { ...item, quantity:
item.quantity + 1 } : item
  ));
};

```

```

const decreaseQty = (id) => {
  setCart(cart.map(item =>
    item.id === id && item.quantity > 1
      ? { ...item, quantity: item.quantity - 1 }
      : item
  ));
};

const removeItem = (id) => {
  setCart(cart.filter(item => item.id !==
id));
};

const totalAmount = cart.reduce(
  (total, item) => total + item.price *
item.quantity,
  0
);

const filteredProducts =
productsData.filter(product =>
  product.name.toLowerCase().includes(search
.toLowerCase())
);

return (
  <div className="container">
    <h1>&img alt="shopping cart icon" data-bbox="598 648 625 665"/> Online Shopping</h1>

    <input
      type="text"
      placeholder="Search product..."
      value={search}
      onChange={(e) =>
setSearch(e.target.value)}
      className="search"
    />

    <div className="products">
      {filteredProducts.map(product => (

```

```

    <div key={product.id}
className="card">
    <img src={product.image}
alt={product.name} />
    <h3>{product.name}</h3>
    <p>₹ {product.price}</p>
    <button onClick={() =>
addToCart(product)}>Add to Cart</button>
    </div>
  )}
</div>

<h2>Cart</h2>

{cart.length === 0 ? (
  <p>Cart is Empty</p>
) : (
  <div>
    {cart.map(item => (
      <div key={item.id}
className="cart-item">
        <span>{item.name}</span>
        <div>
          <button onClick={() =>
decreaseQty(item.id)}>-</button>
          <span>{item.quantity}</span>
          <button onClick={() =>
increaseQty(item.id)}>+</button>
        </div>
          <span>₹ {item.price *
item.quantity}</span>
          <button onClick={() =>
removeItem(item.id)}>Remove</button>
        </div>
      )}
    )}

    <h3>Total: ₹ {totalAmount}</h3>

    <button className="checkout-btn"
onClick={() => setShowCheckout(true)}>
      Proceed to Checkout

```

```

    </button>
  </>
)}

{showCheckout && (
  <div className="checkout">
    <h2>Checkout</h2>
    <input type="text" placeholder="Full
Name" />
    <input type="text"
placeholder="Address" />
    <button onClick={() => alert("Order
Placed Successfully!")}>
      Place Order
    </button>
  </div>
)}
</div>
);
}

```

export default App;

App.css:

```

body {
  font-family: Arial, sans-serif;
  background-color: #d0bee4;
}

.container {
  padding: 20px;
}

.search {
  padding: 8px;
  margin-bottom: 20px;
  width: 200px;
}

.products {
  display: flex;

```

```

    gap: 20px;
    margin-bottom: 30px;
  }

  .card {
    background: white;
    padding: 15px;
    border-radius: 10px;
    box-shadow: 0 4px 10px rgba(0,0,0,0.1);
    width: 180px;
    text-align: center;
  }

  .card img {
    width: 100%;
    border-radius: 5px;
  }

  .card button {
    background-color: #8b55a4;
    color: white;
    border: none;
    padding: 8px;
    border-radius: 5px;
    cursor: pointer;
  }

  .card button:hover {
    background-color: #0056b3;
  }

  .cart-item {
    display: flex;
    justify-content: space-between;
    align-items: center;
    margin-bottom: 10px;
  }

  .cart-item button {
    margin-left: 5px;
    padding: 4px 8px;

```

```

  }

  .checkout-btn {
    margin-top: 10px;
    background-color: green;
    color: white;
    padding: 8px 12px;
    border: none;
    border-radius: 5px;
  }

  .checkout {
    margin-top: 20px;
    background: white;
    padding: 15px;
    border-radius: 10px;
  }

  .checkout input {
    display: block;
    margin-bottom: 10px;
    padding: 6px;
    width: 200px;
  }

```

Run Frontend:

npm start

Output:

